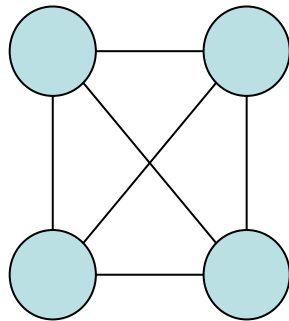


Data Structures

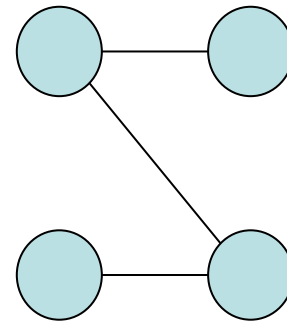
25. Minimum Spanning Tree (MST)

Spanning Trees

- A spanning tree of a graph is just a subgraph that contains all the vertices and is a tree
- Formal definition
 - Given a connected graph with $|V| = n$ vertices
 - A spanning tree is defined a collection of $n - 1$ edges which connect all n vertices
 - The n vertices and $n - 1$ edges define a connected sub-graph



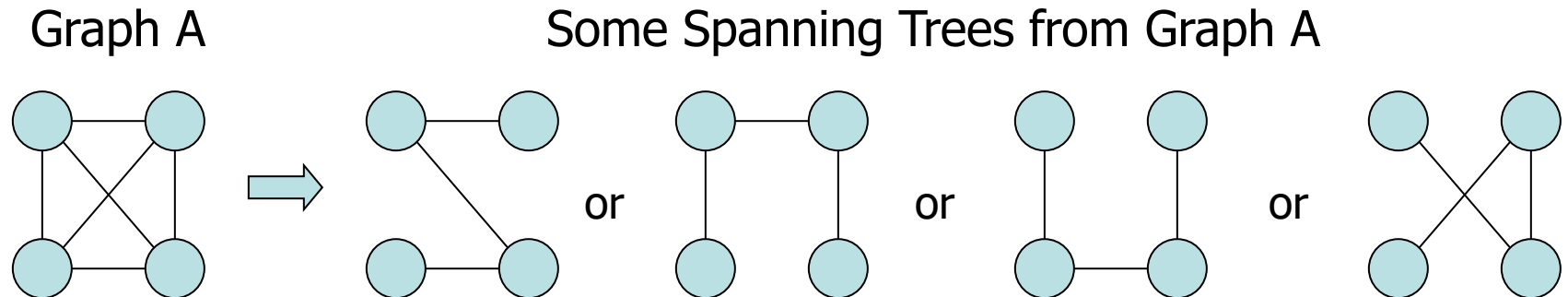
Graph



Spanning Tree

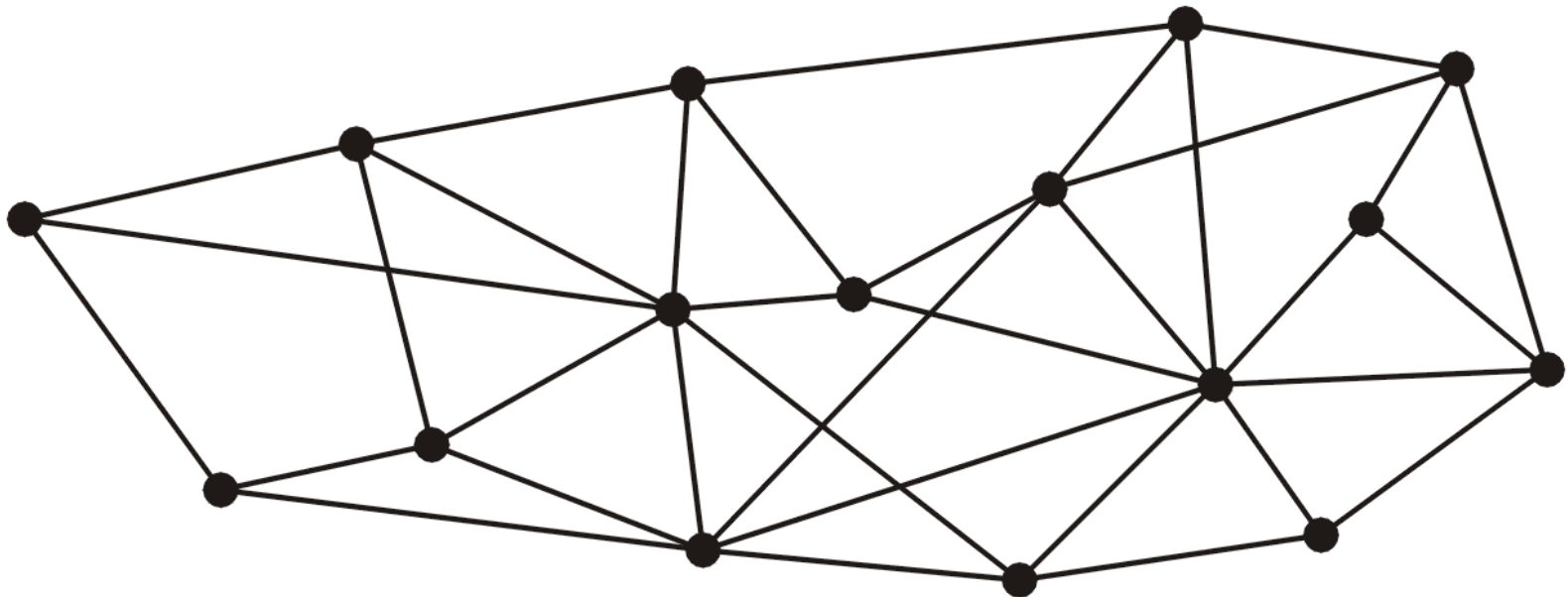
Spanning Trees

- A spanning tree is not necessarily unique



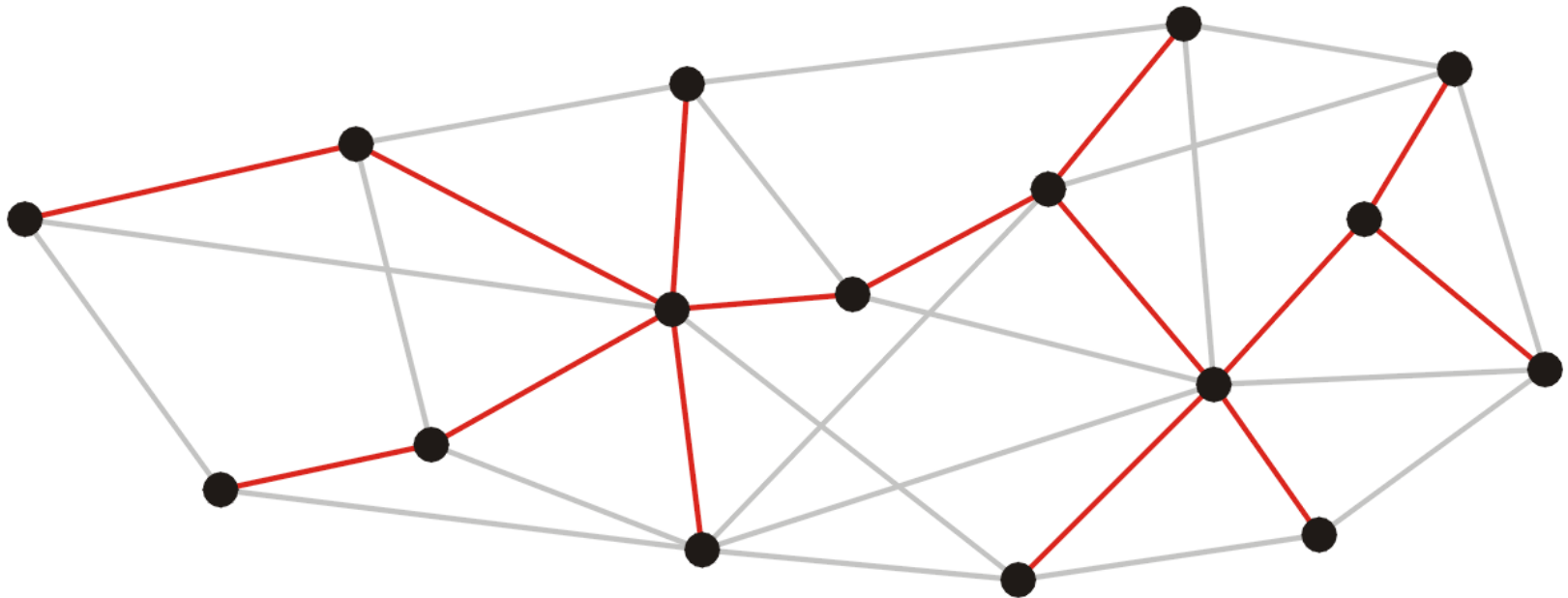
Spanning Trees – Example

- This graph has 16 vertices and 35 edges



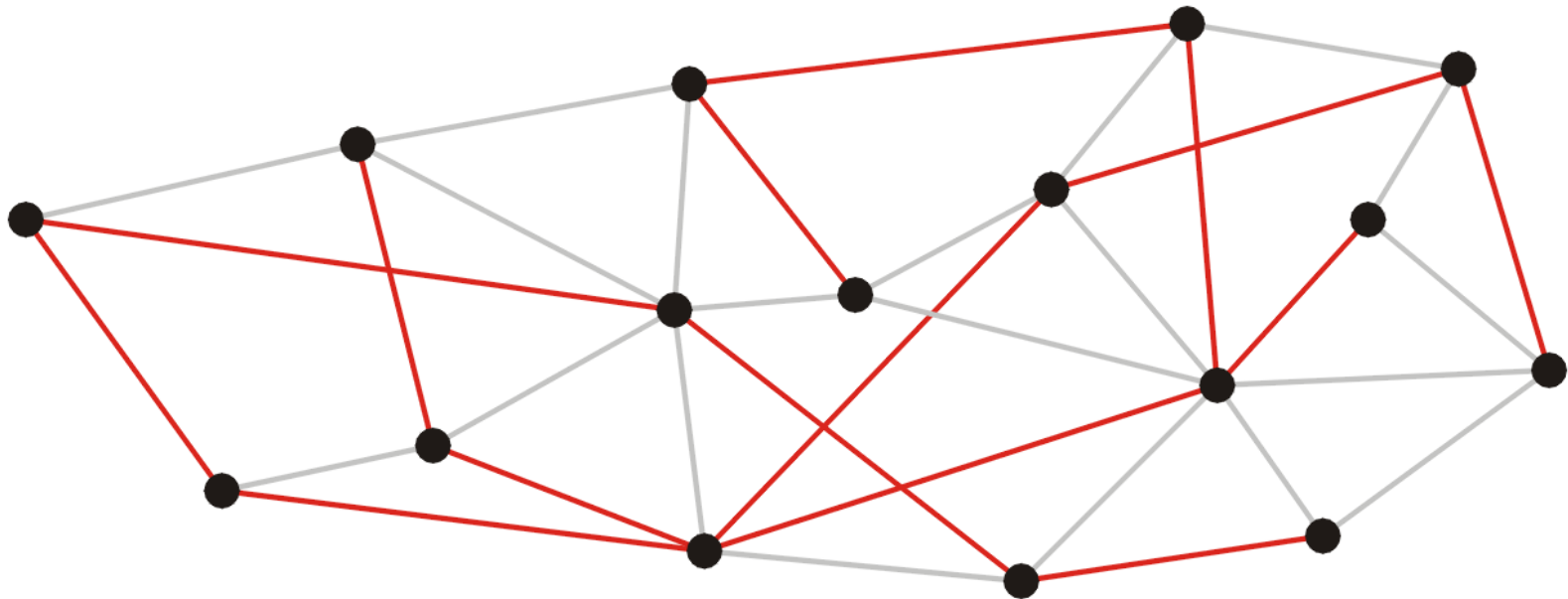
Spanning Trees – Example

- These 15 edges form a minimum spanning tree

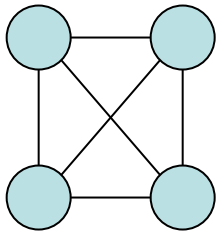


Spanning Trees – Example

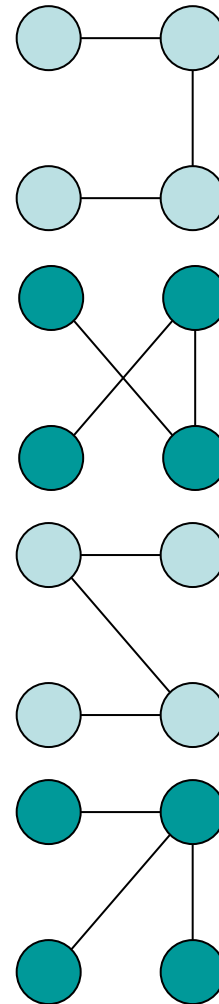
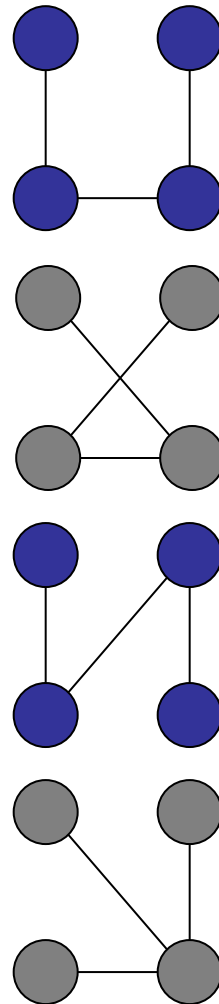
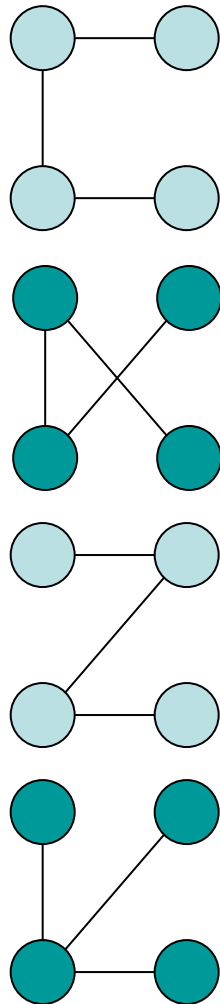
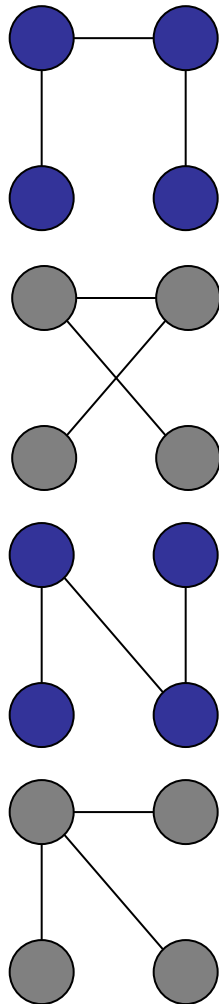
- As do these 15 edges



Spanning Trees – Example



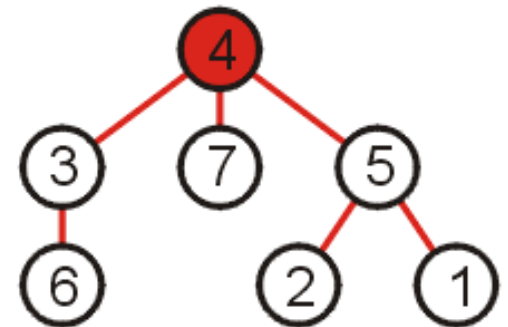
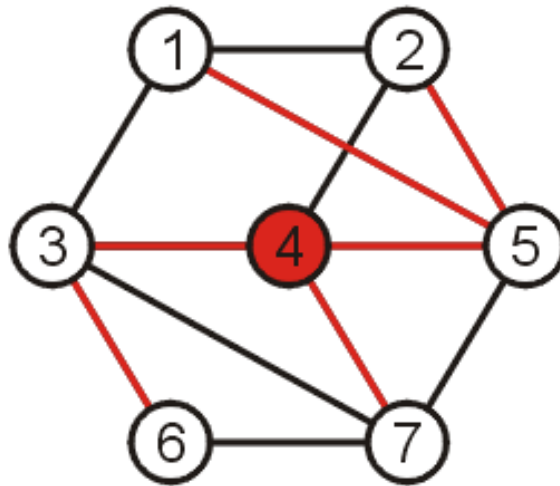
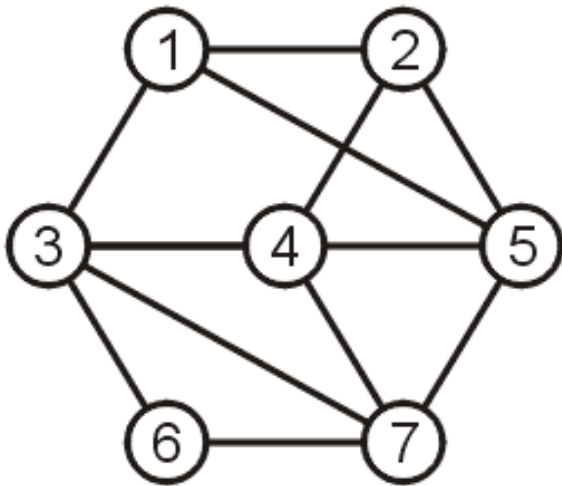
Graph



All 16 of its Spanning Trees

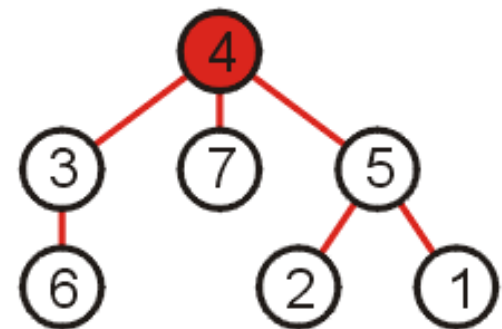
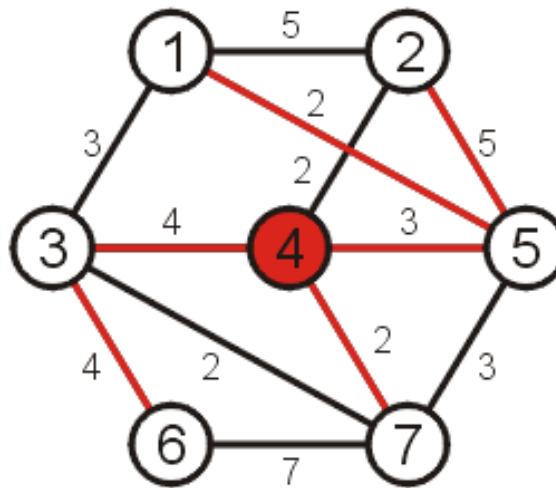
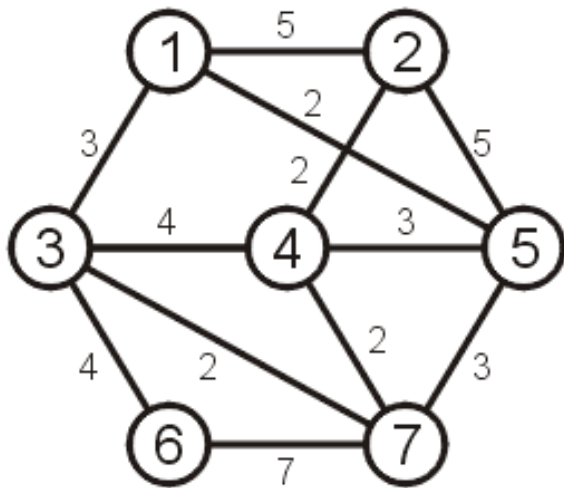
Spanning Trees

- Why such a collection of $|V| - 1$ edges is called a tree?
 - If any vertex is taken to be the root, we form a tree by treating the adjacent vertices as children, and so on...



Spanning Tree on Weighted Graphs

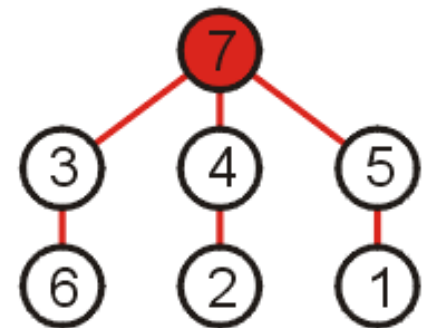
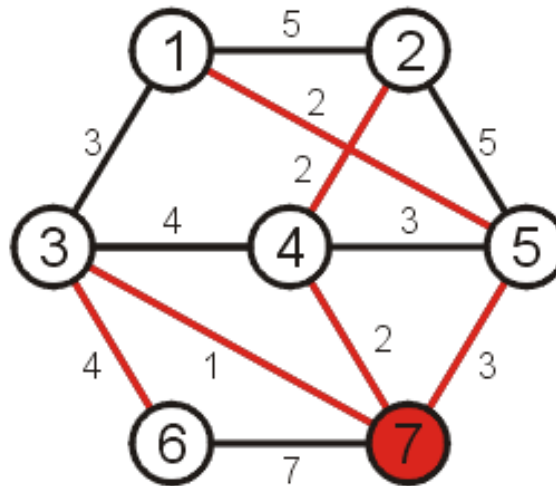
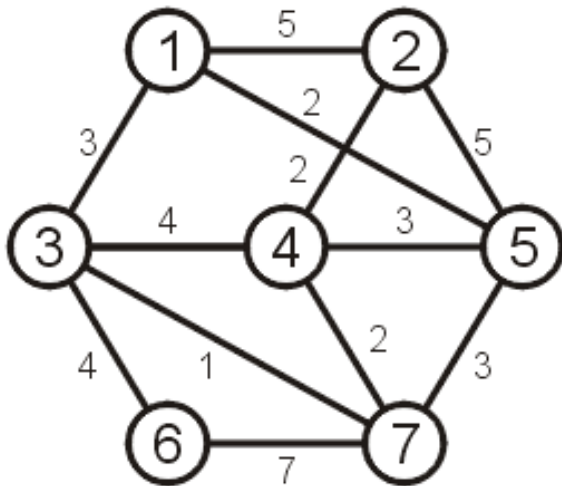
- Weight of a spanning tree
 - Sum of the weights on all the edges which comprise the spanning tree



- The weight of this spanning tree is 20

Minimum Spanning Tree (MST)

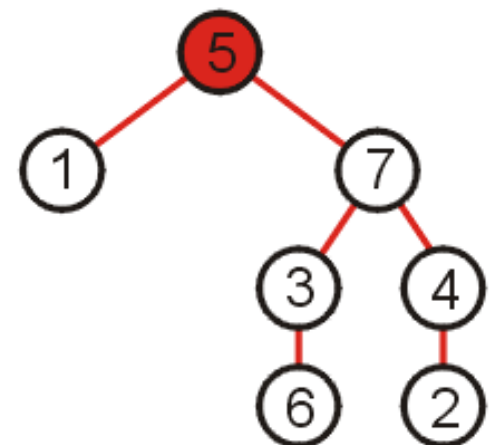
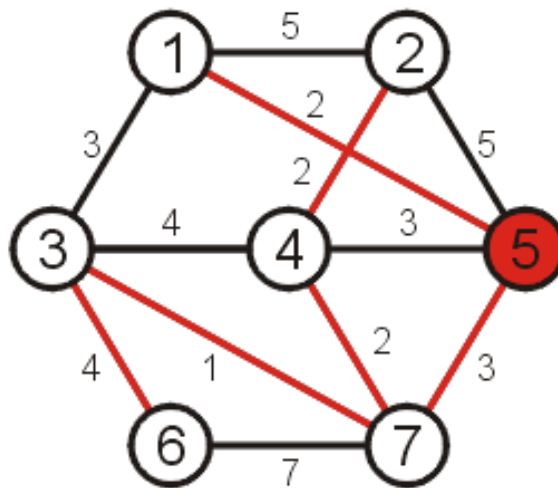
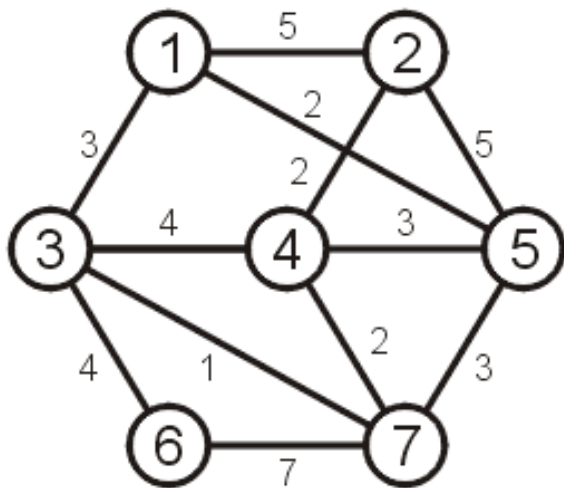
- Spanning tree that minimizes the weight
 - Such a tree is termed a minimum spanning tree



- The weight of this spanning tree is 14

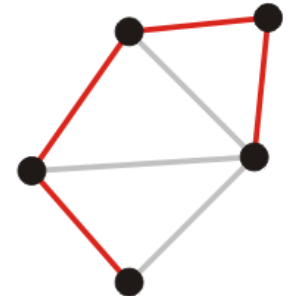
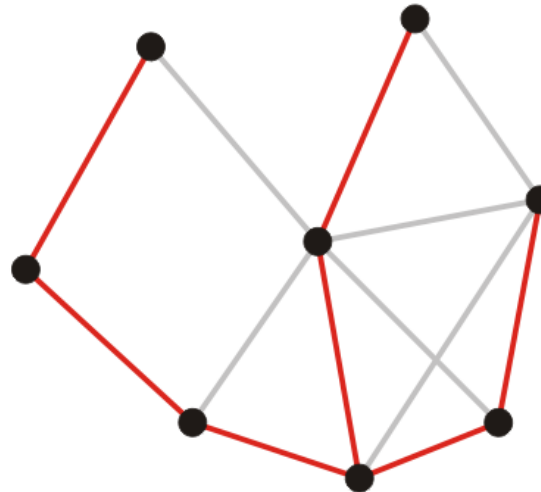
Minimum Spanning Tree (MST)

- If a different vertex is used as the root
 - A different tree is obtained
 - However, this is simply the result of one or more rotations



Spanning Forest

- Suppose that a graph is composed of N connected sub-graphs
- A spanning forest is a collection of N spanning trees
 - One for each connected sub-graph



- A minimum spanning forest
 - A collection of N minimum spanning trees
 - One for each connected vertex-induced sub-graph

Algorithms For Obtaining MST

- Kruskal's Algorithm
- Prim's Algorithm
- Boruvka's Algorithm

Kruskal's Algorithm

Kruskal's Algorithm

- Kruskal's algorithm creates a forest of trees
- Initially forest consists of N single node trees (and no edges)
- Sorts the edges by weight and goes through the edges from least weight to greatest weight
- At each step one edge (with least weight) is added so that it joins two trees together
 - As long as the addition does not create a cycle

Kruskal's Algorithm

The halting conditions are as follows:

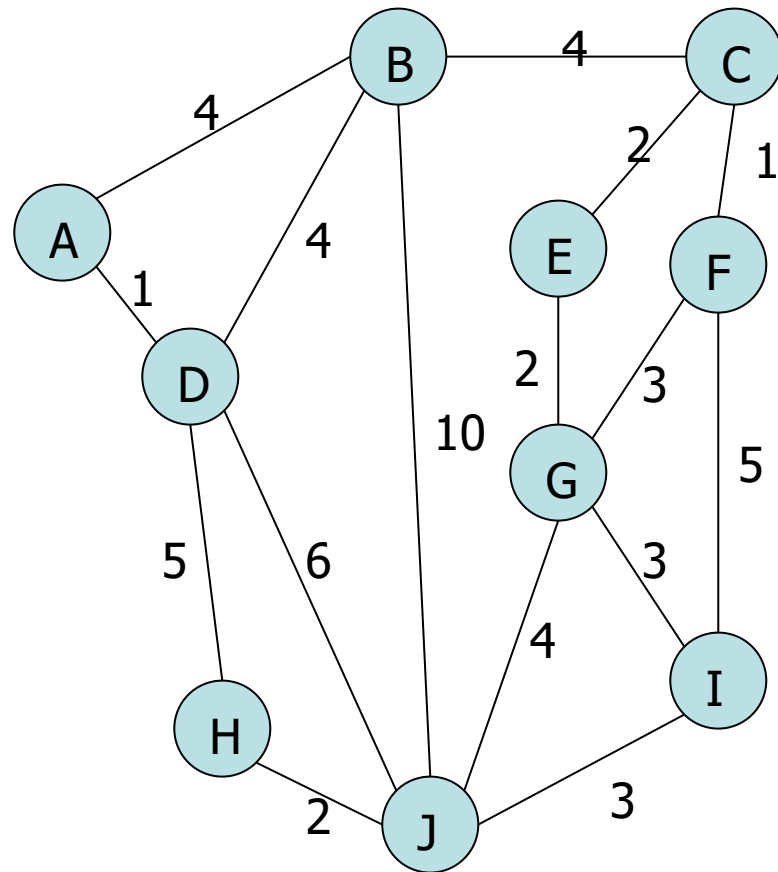
1. When $|V| - 1$ edges have been added
 - In this case we have a minimum spanning tree
2. We have gone through all edges
 - A forest of minimum spanning trees on all connected sub-graphs

Kruskal's Algorithm

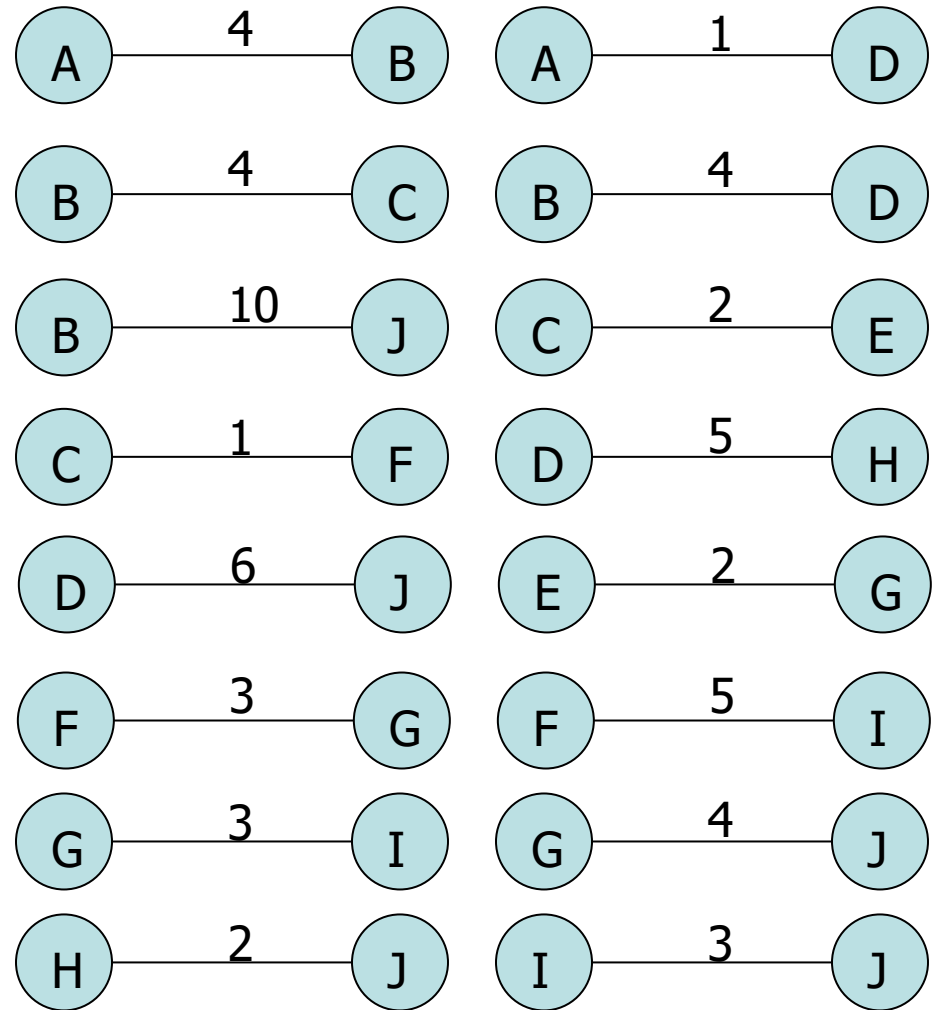
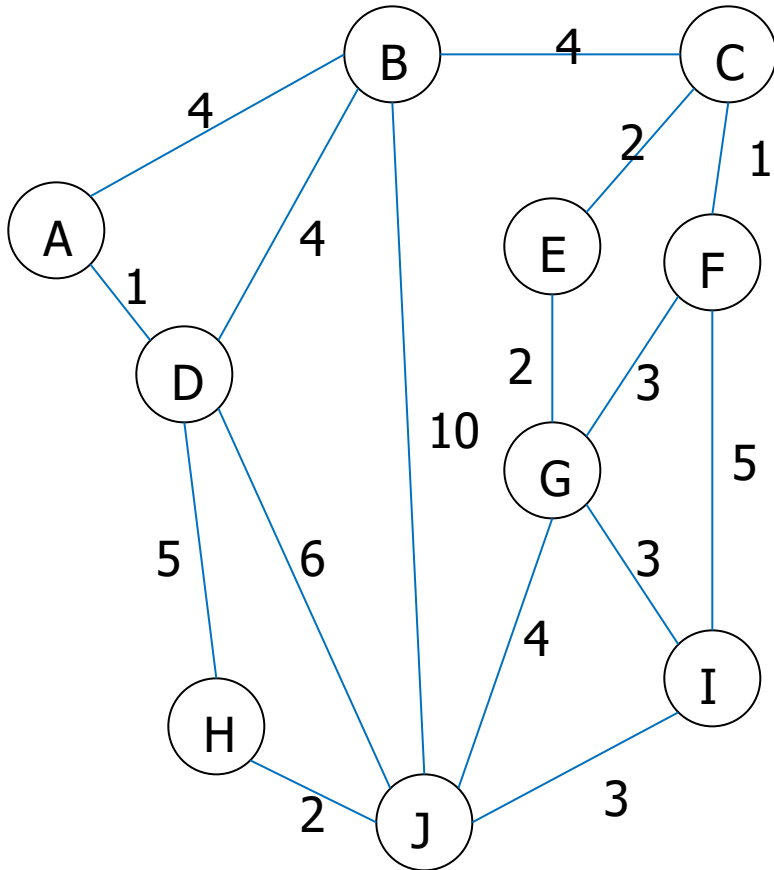
1. The forest is constructed – with each node in a separate tree
 2. The edges are placed in a priority queue
 3. Until we've added $n-1$ edges (assumption: connected graph)
 1. Extract the cheapest edge from the queue
 2. If it forms a cycle, reject it
 3. Else add it to the forest. Adding it to the forest will join two trees
- Every step will have joined two trees in the forest together, so that at the end, there will only be one tree

Kruskal's Algorithm – Example

- Complete Graph

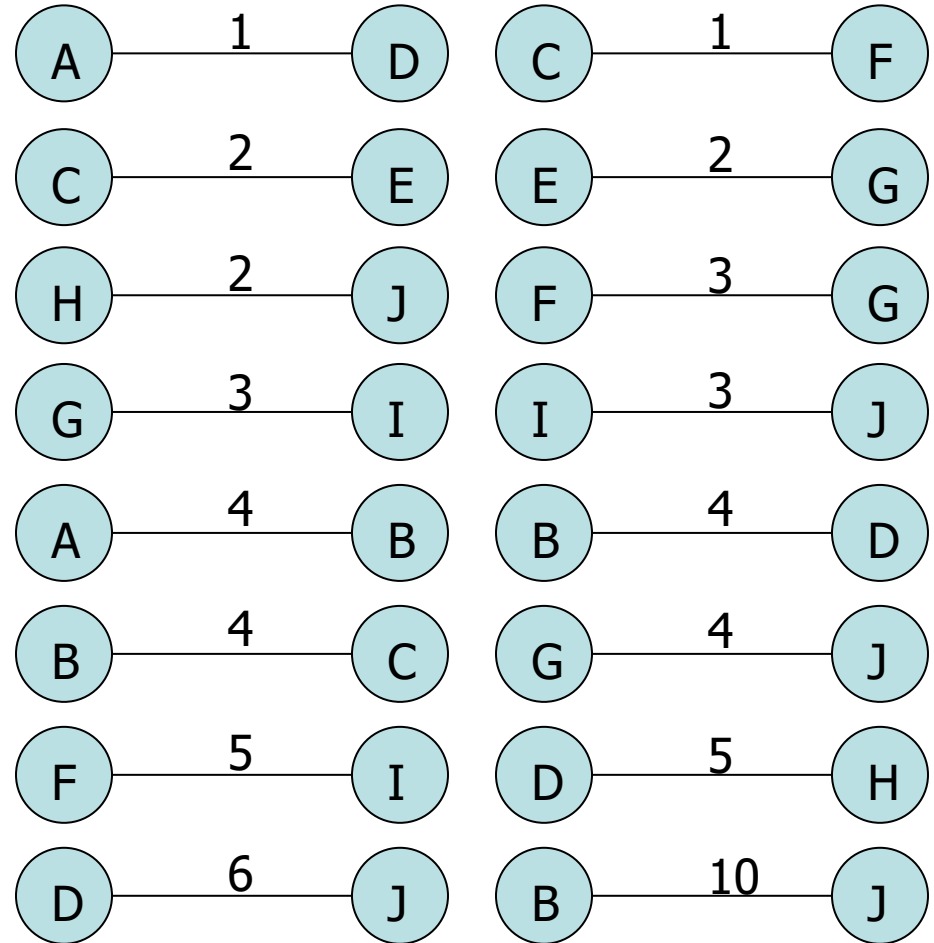
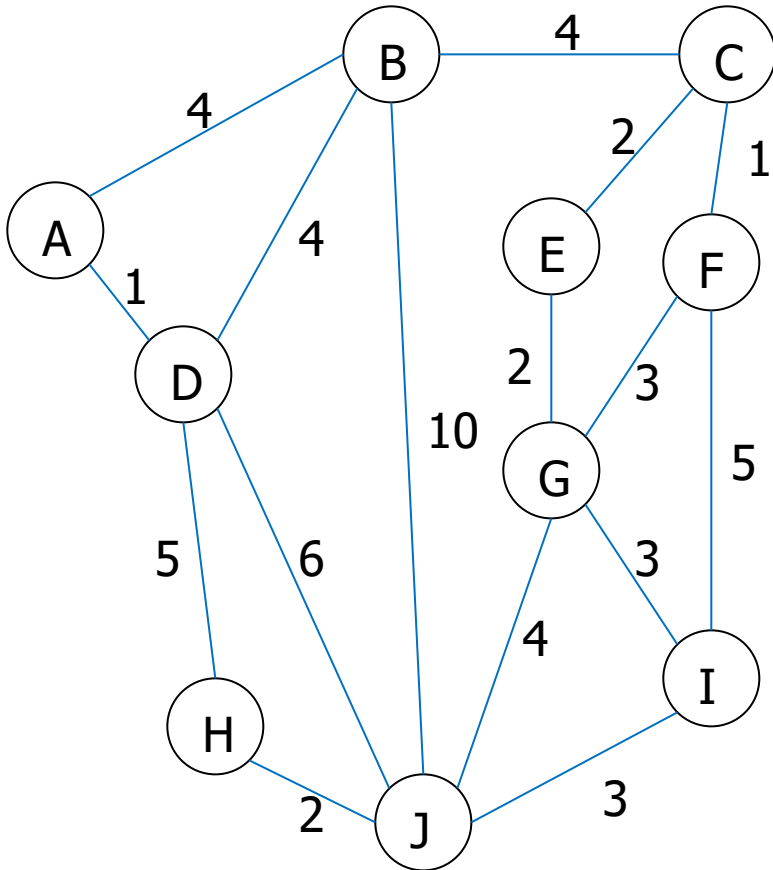


Kruskal's Algorithm – Example



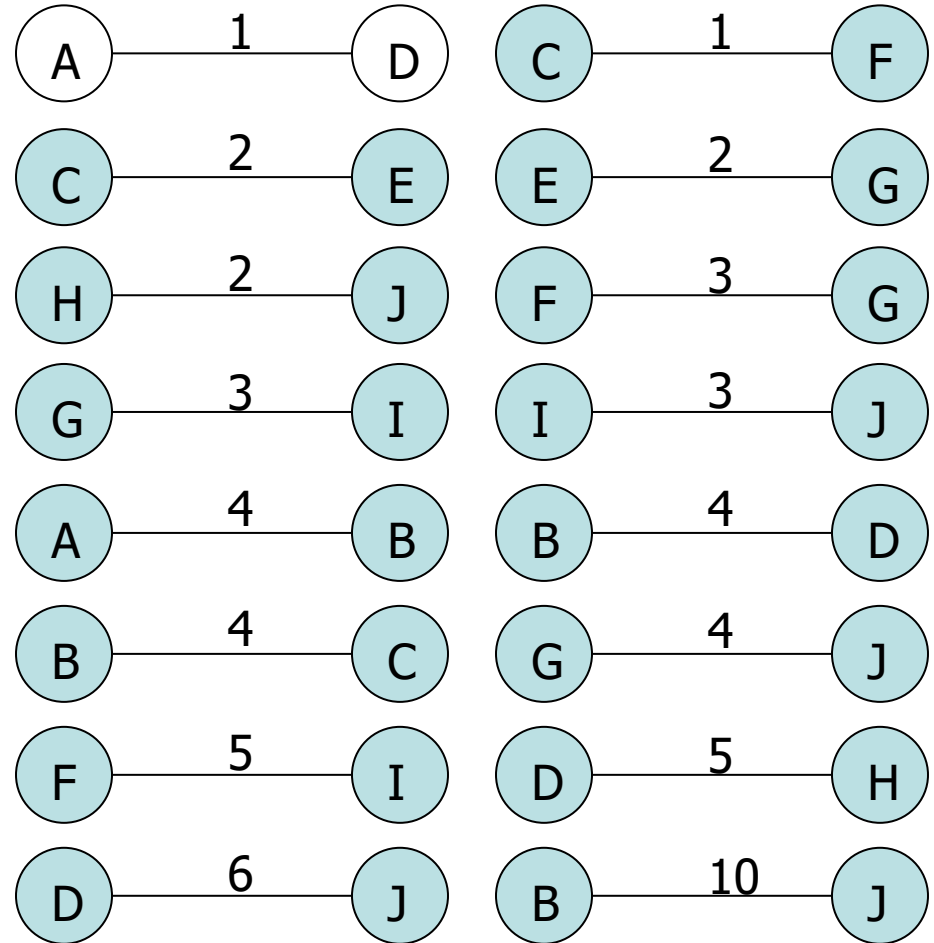
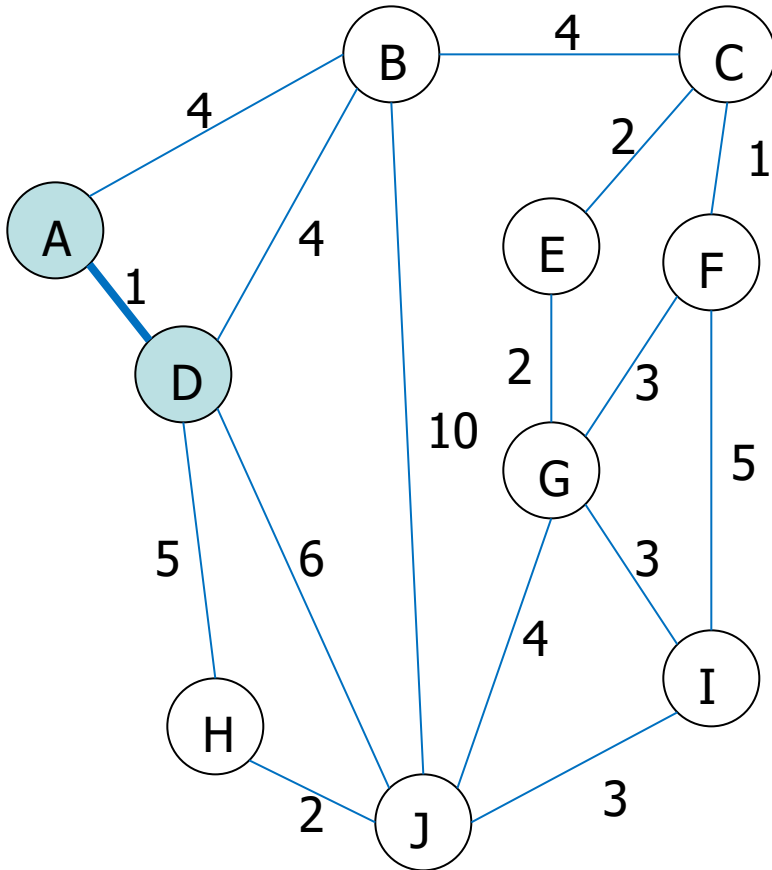
Kruskal's Algorithm – Example

- Sort Edges: In reality priority queue is used



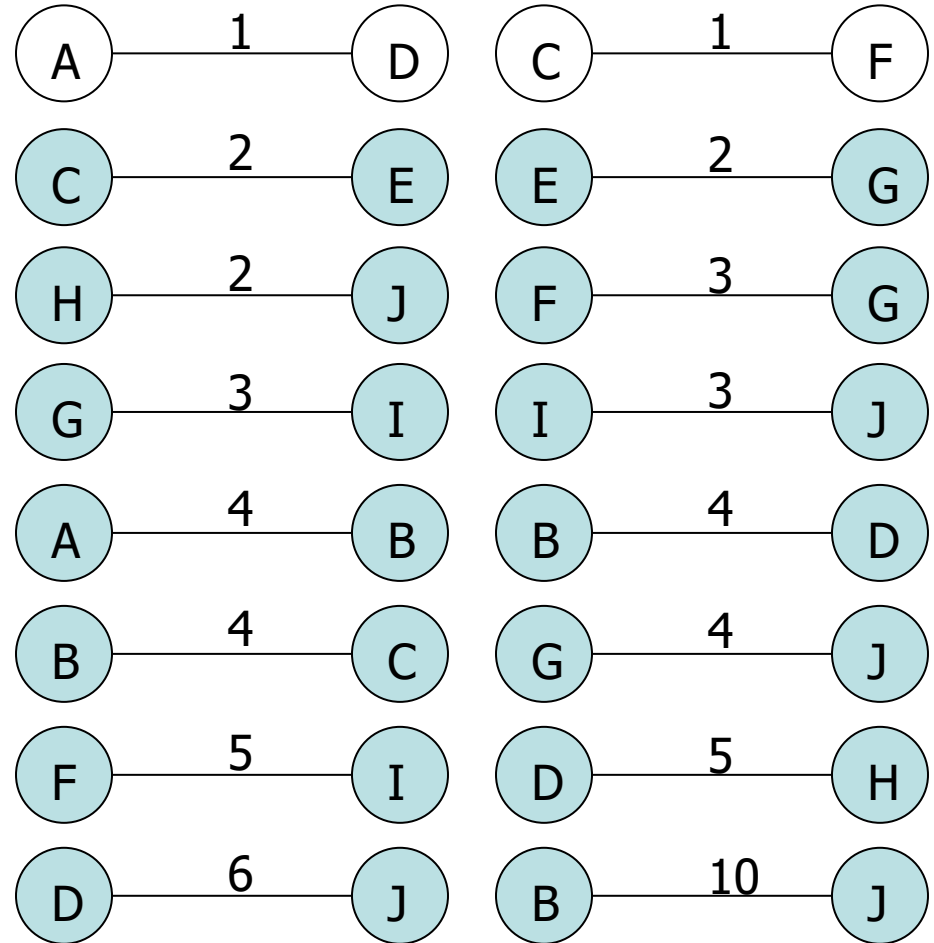
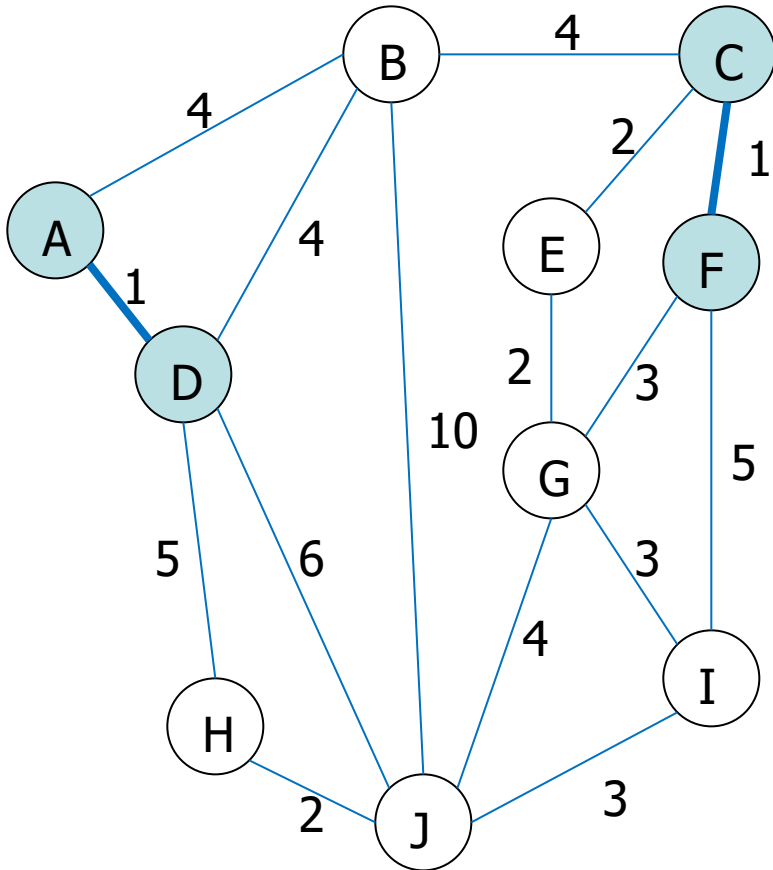
Kruskal's Algorithm – Example

- Add Edge



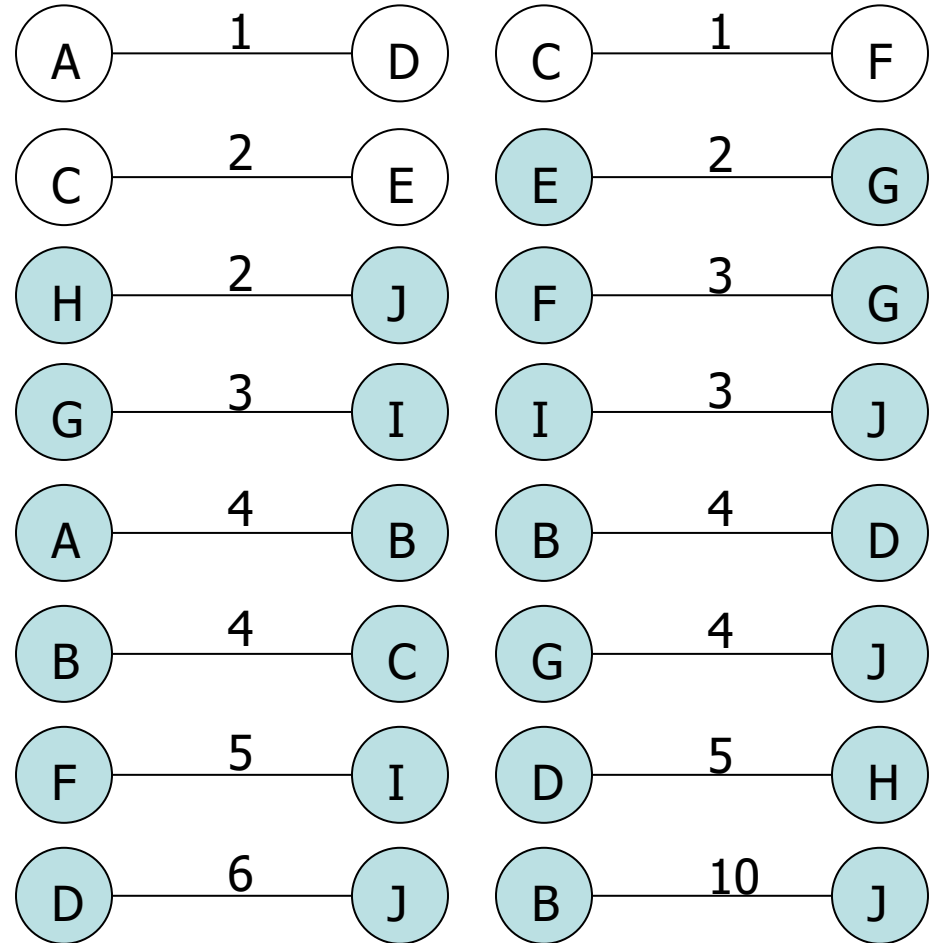
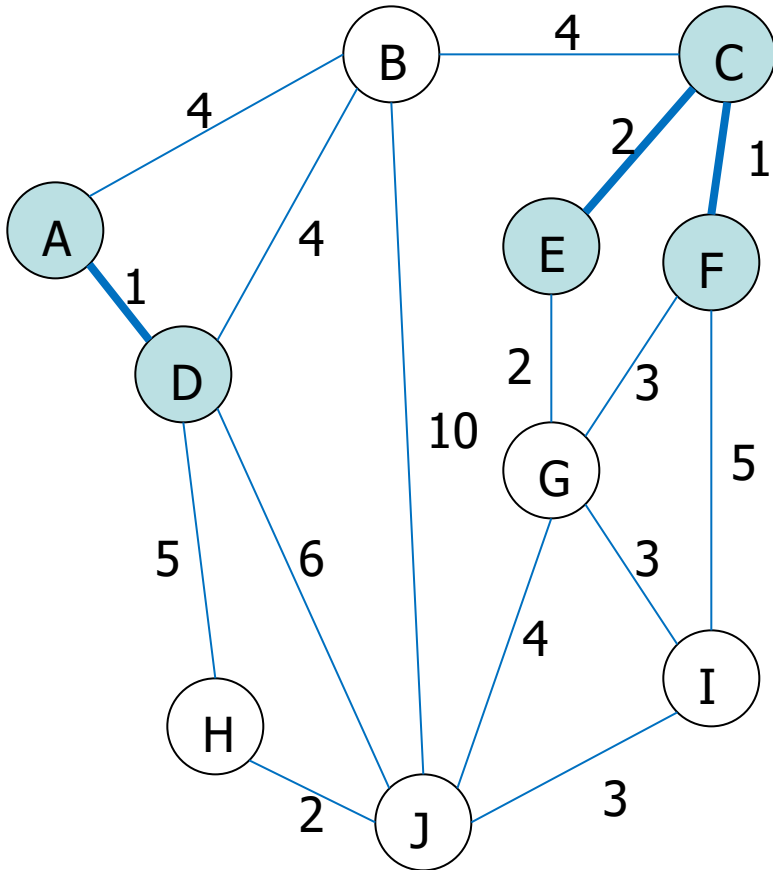
Kruskal's Algorithm – Example

- Add Edge



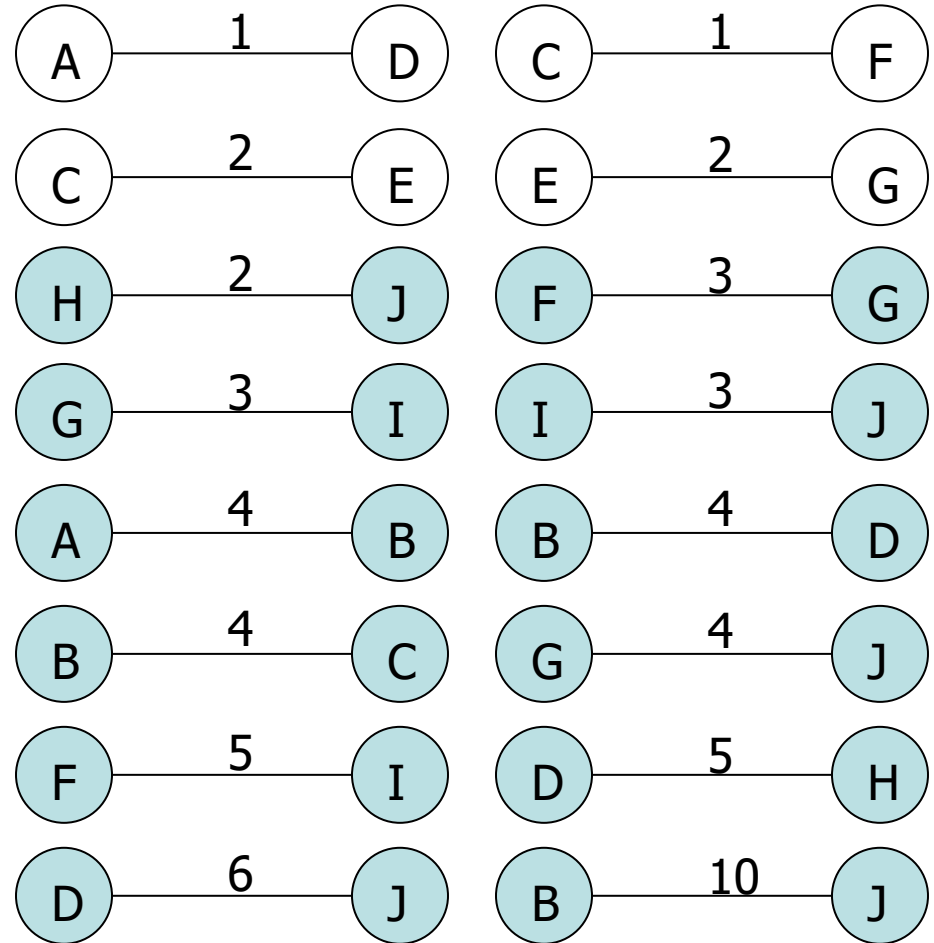
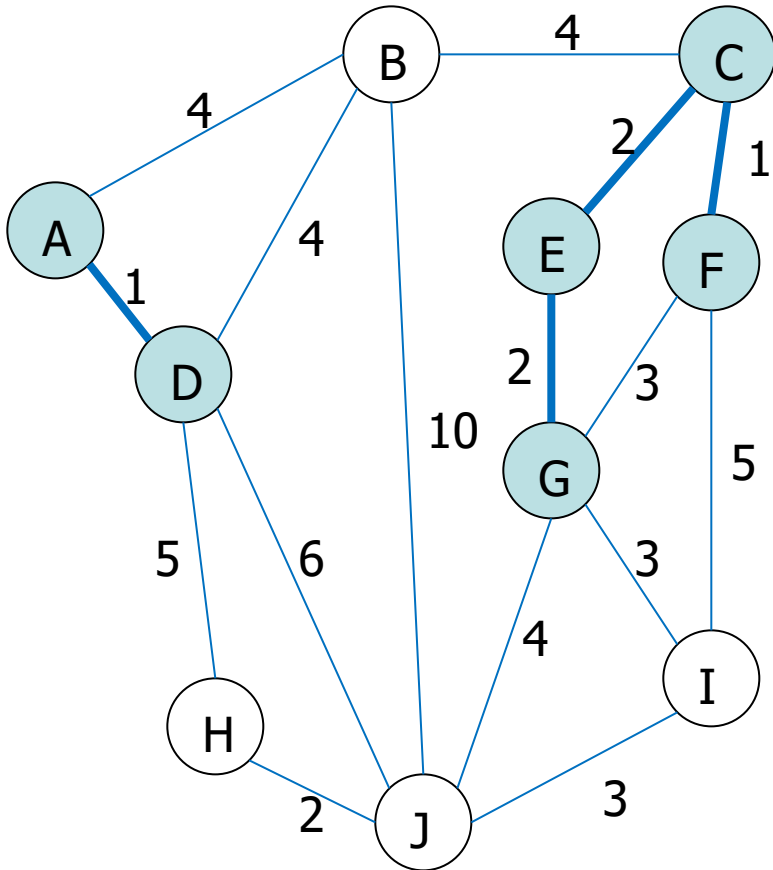
Kruskal's Algorithm – Example

- Add Edge



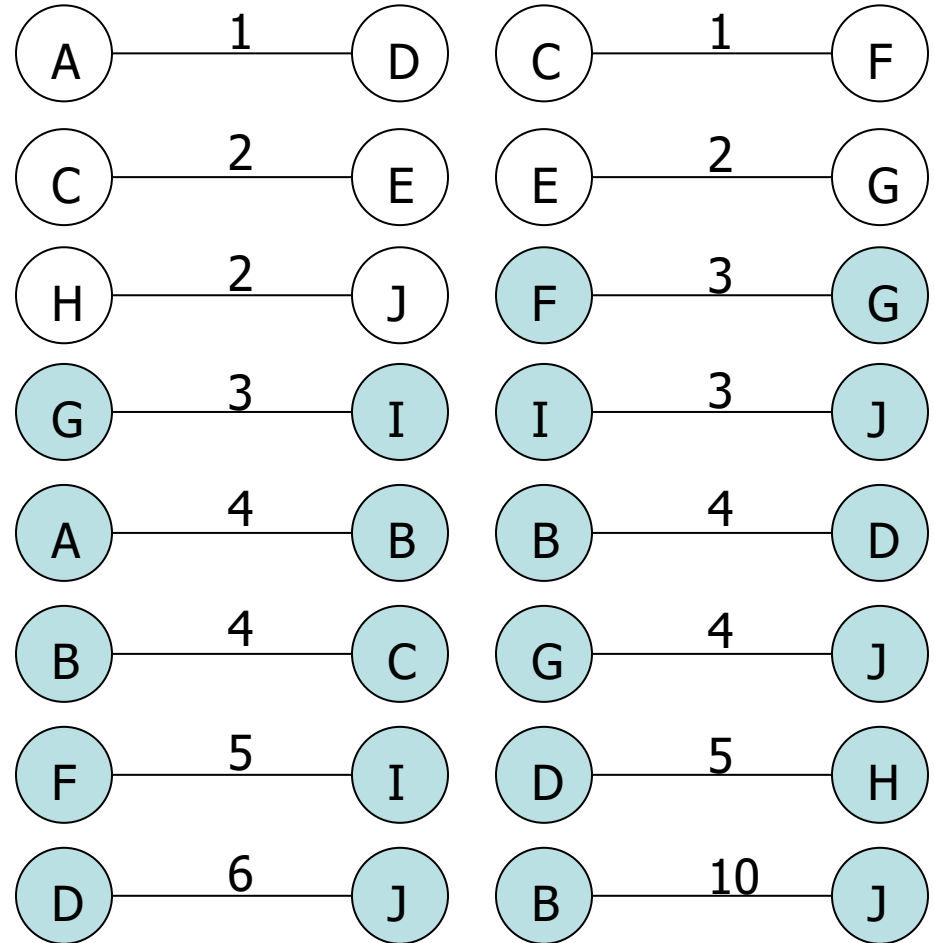
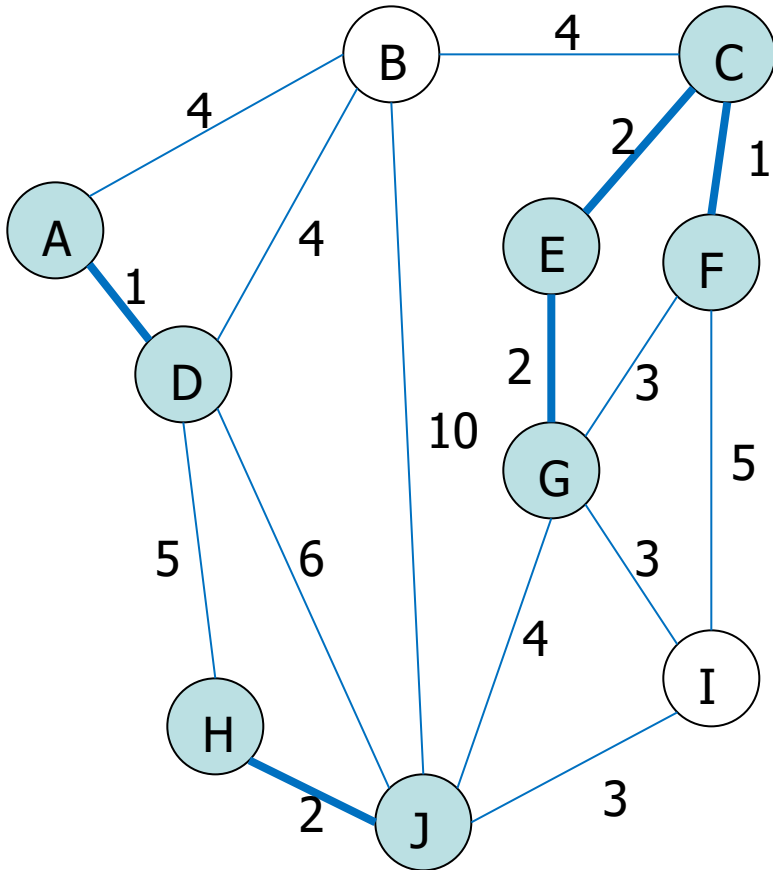
Kruskal's Algorithm – Example

- Add Edge



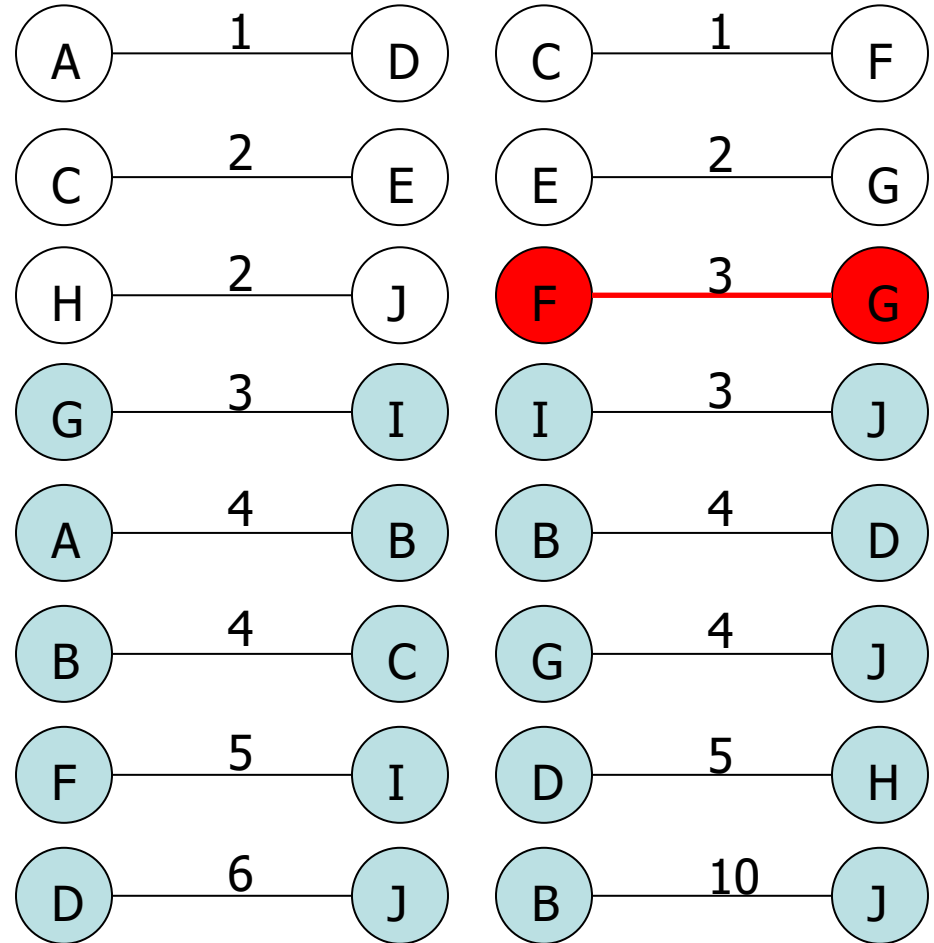
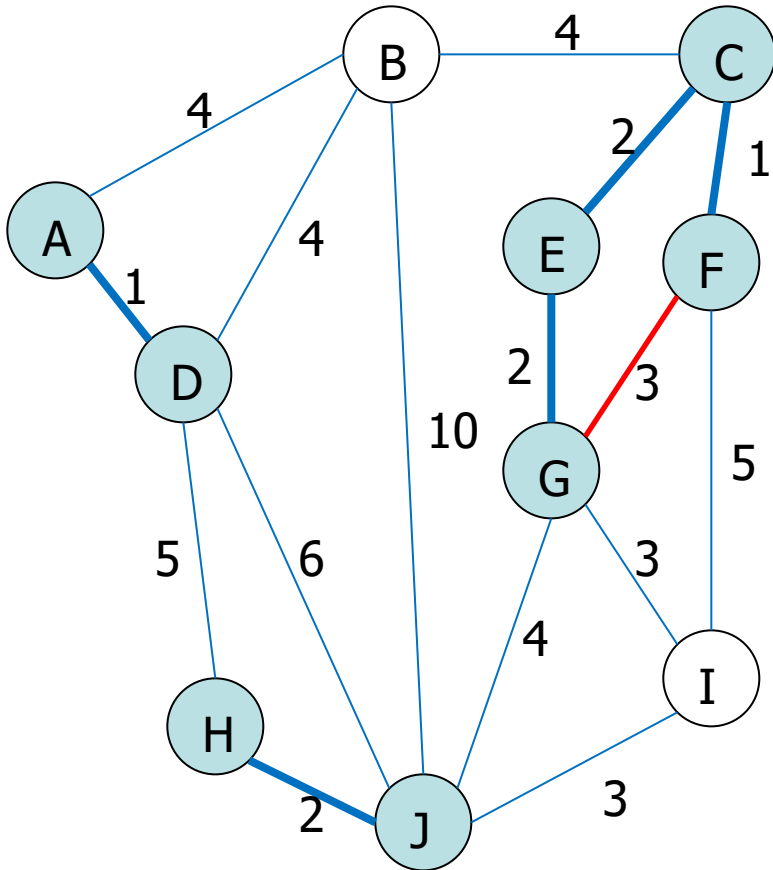
Kruskal's Algorithm – Example

- Add Edge



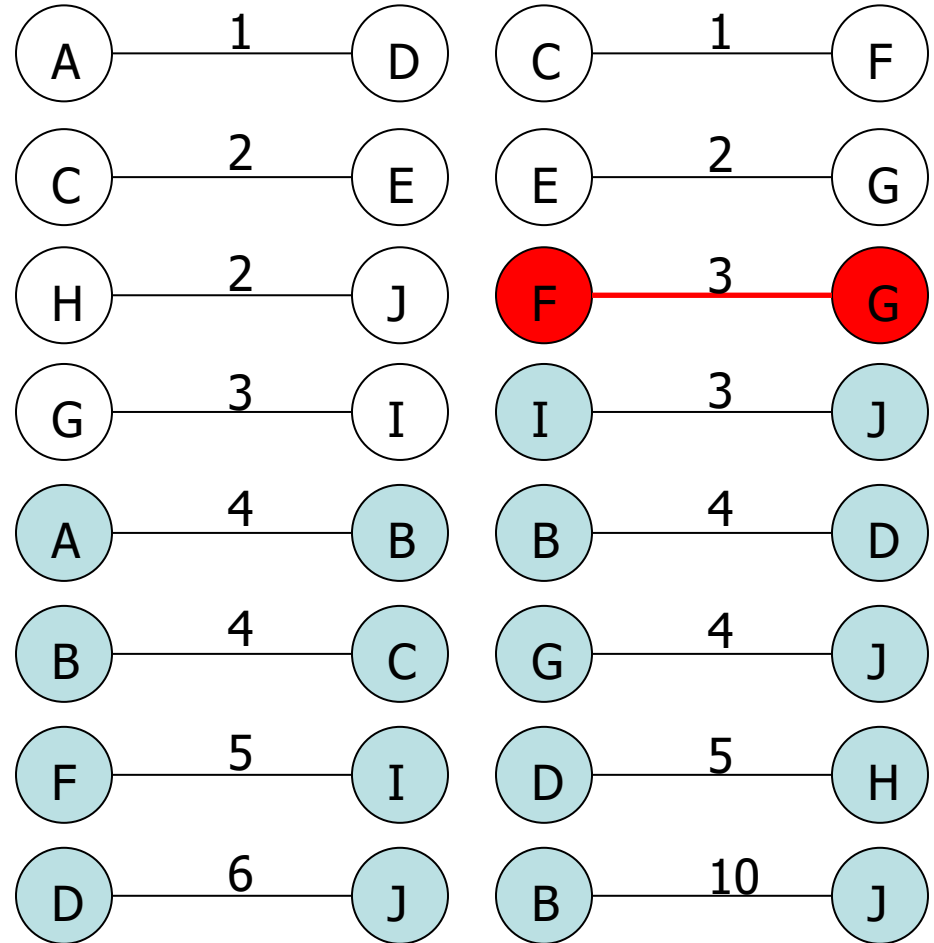
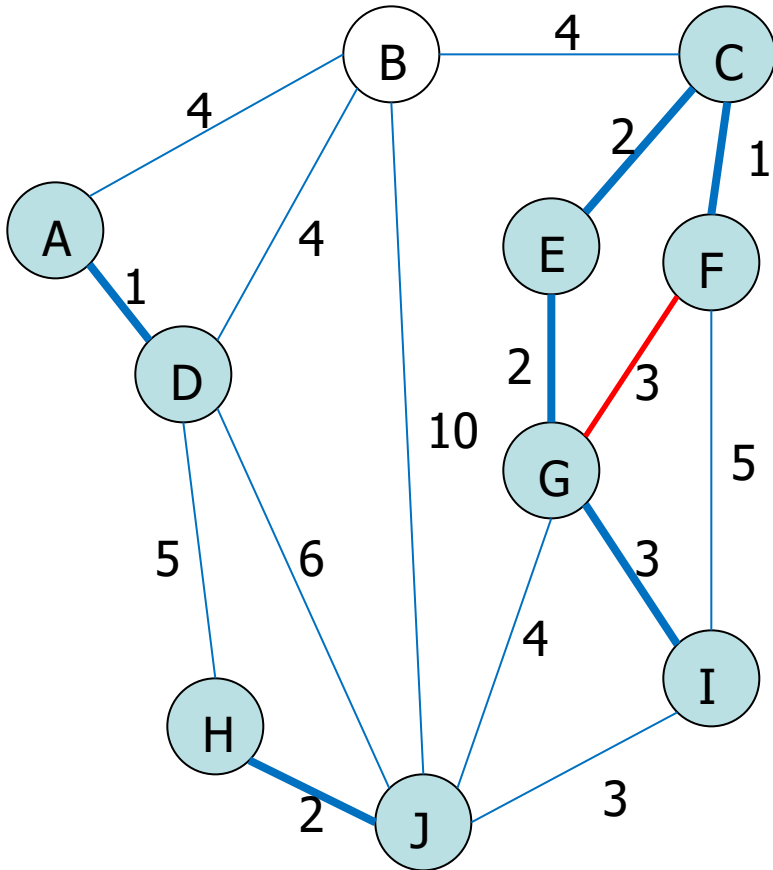
Kruskal's Algorithm – Example

- Add Edge



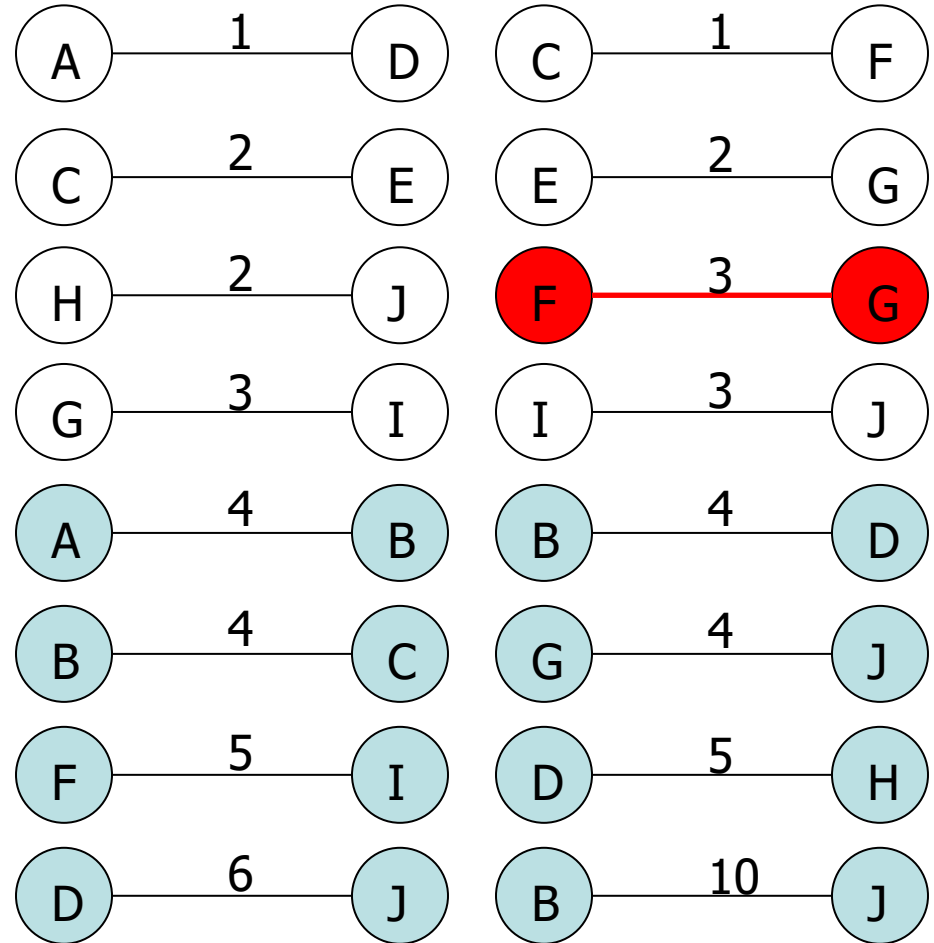
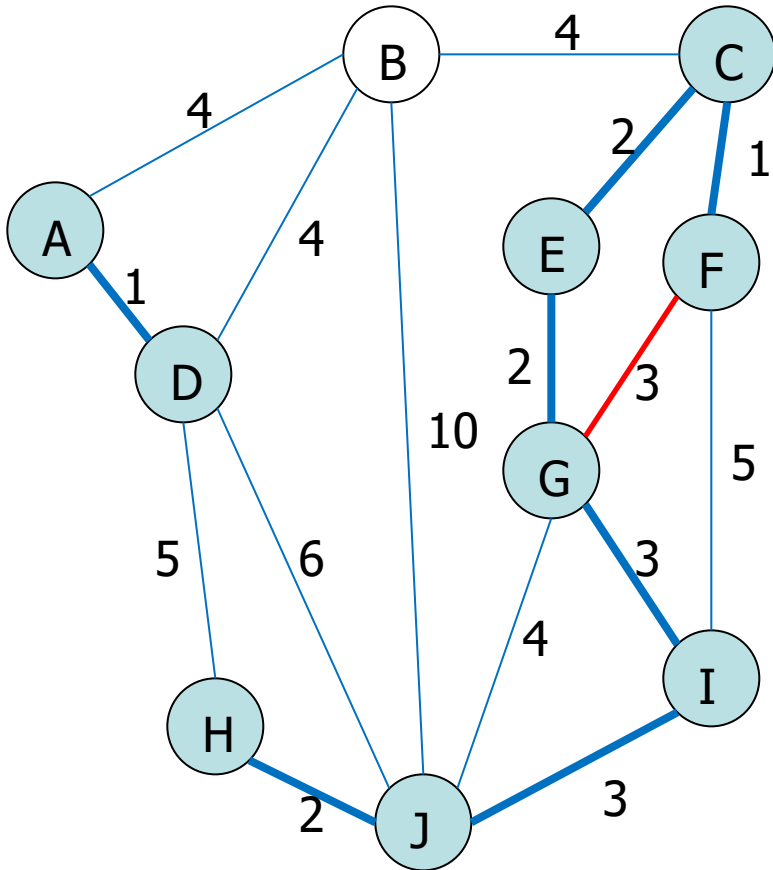
Kruskal's Algorithm – Example

- Add Edge



Kruskal's Algorithm – Example

- Add Edge

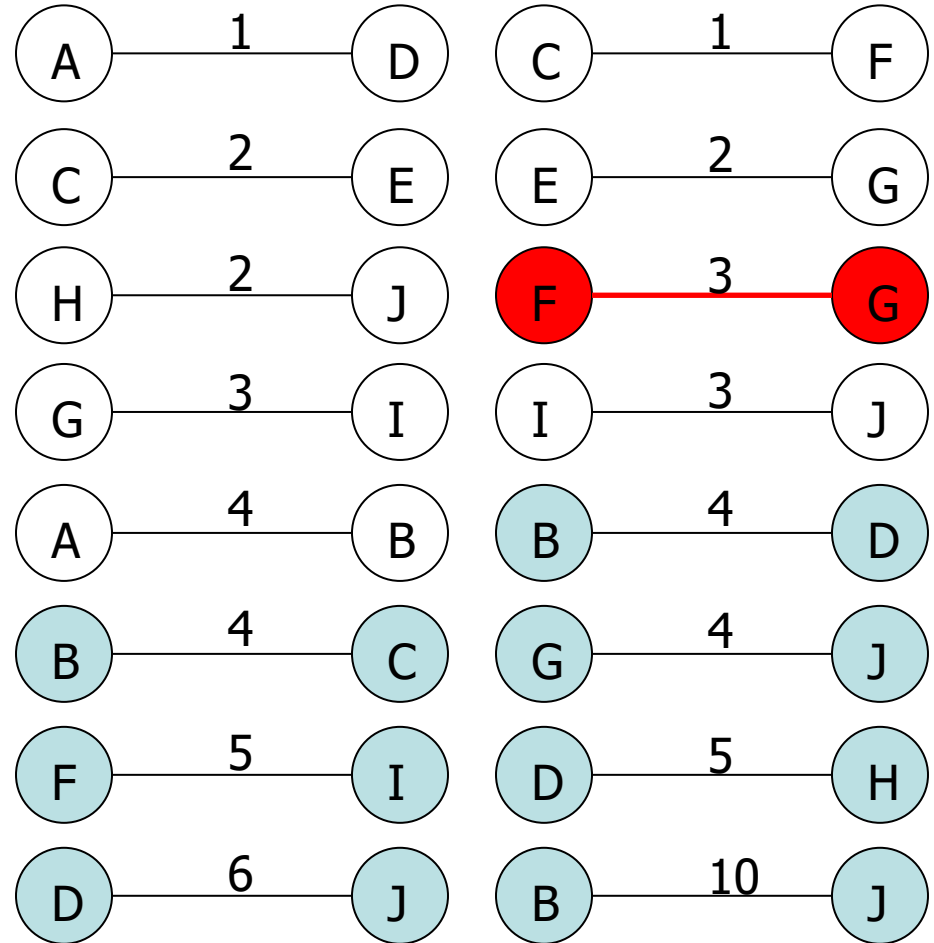
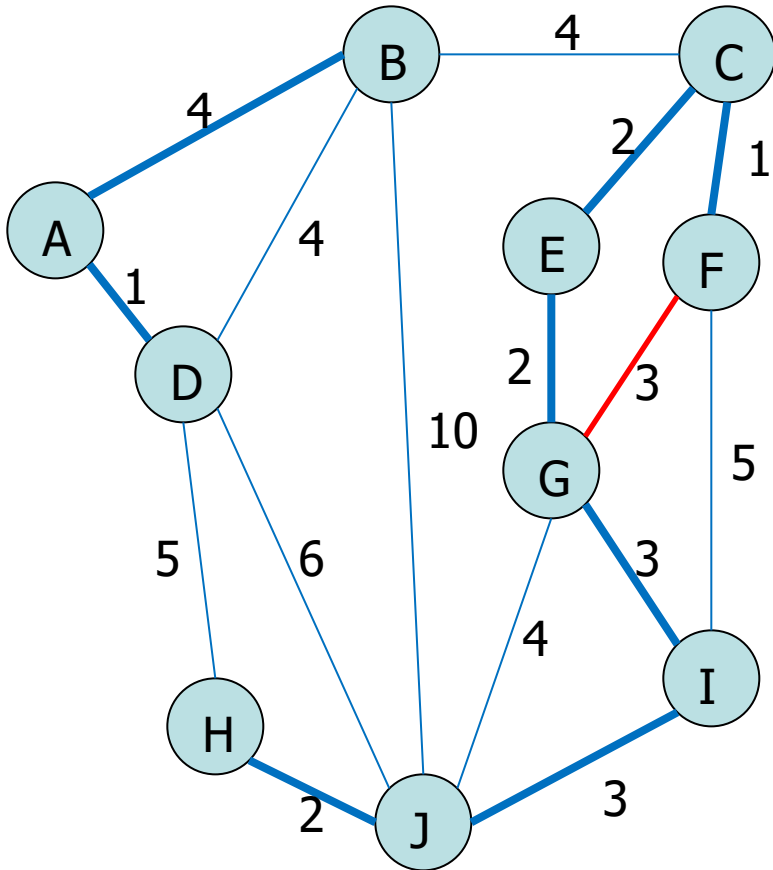


25-MST

28

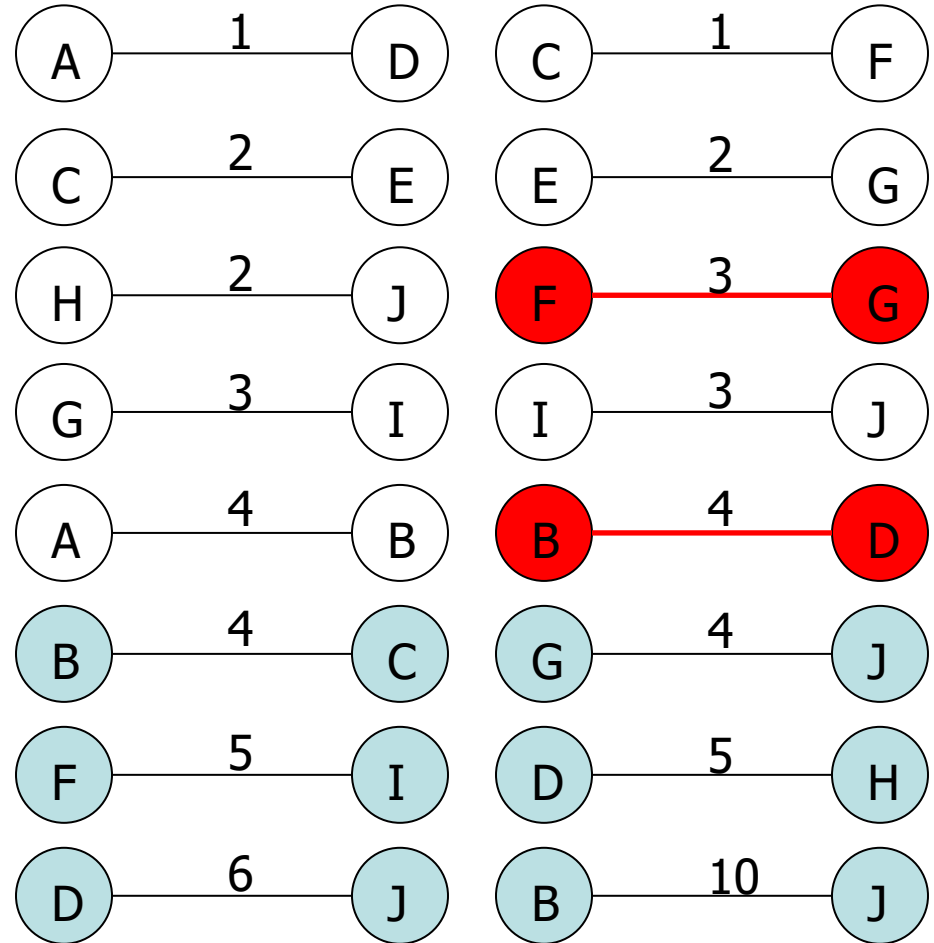
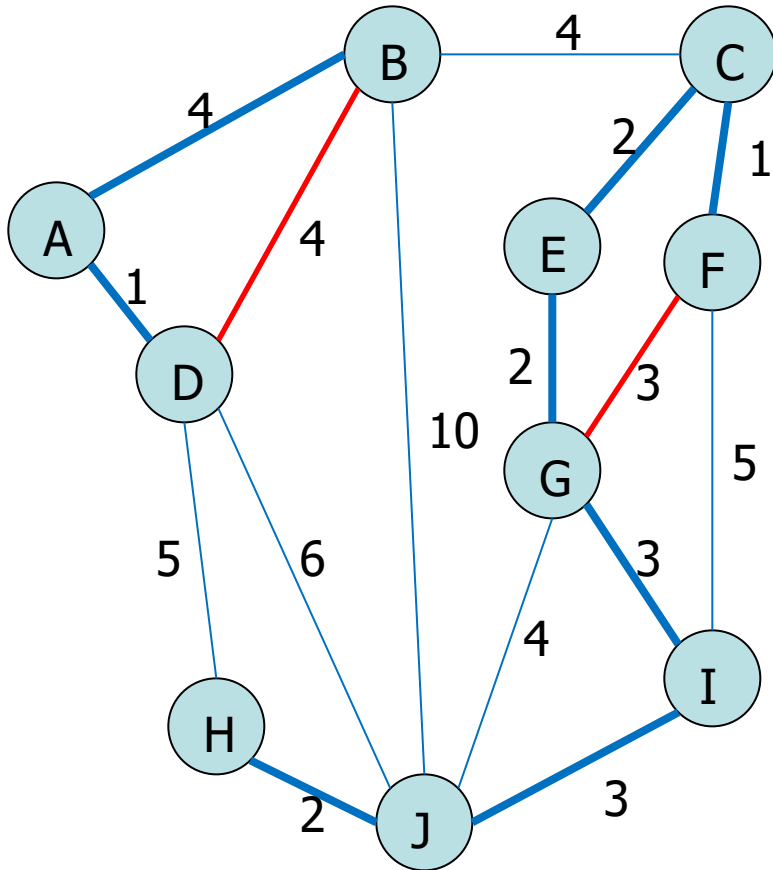
Kruskal's Algorithm – Example

- Add Edge



Kruskal's Algorithm – Example

- Add Edge

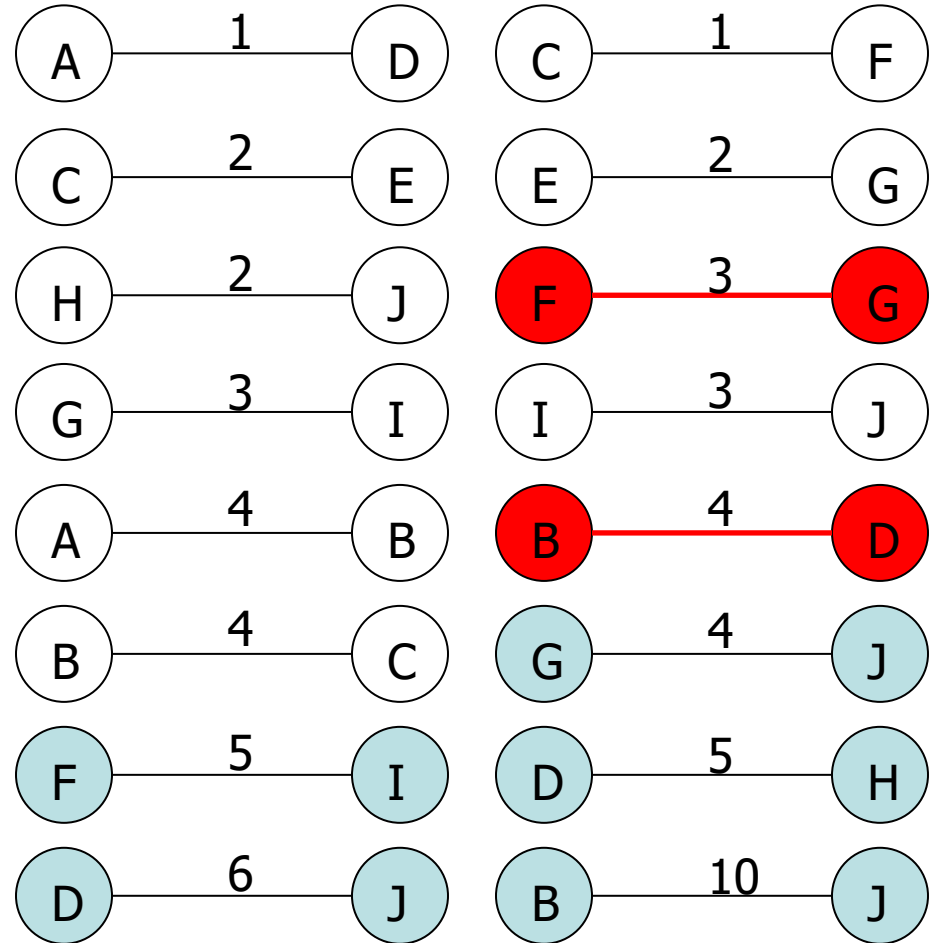
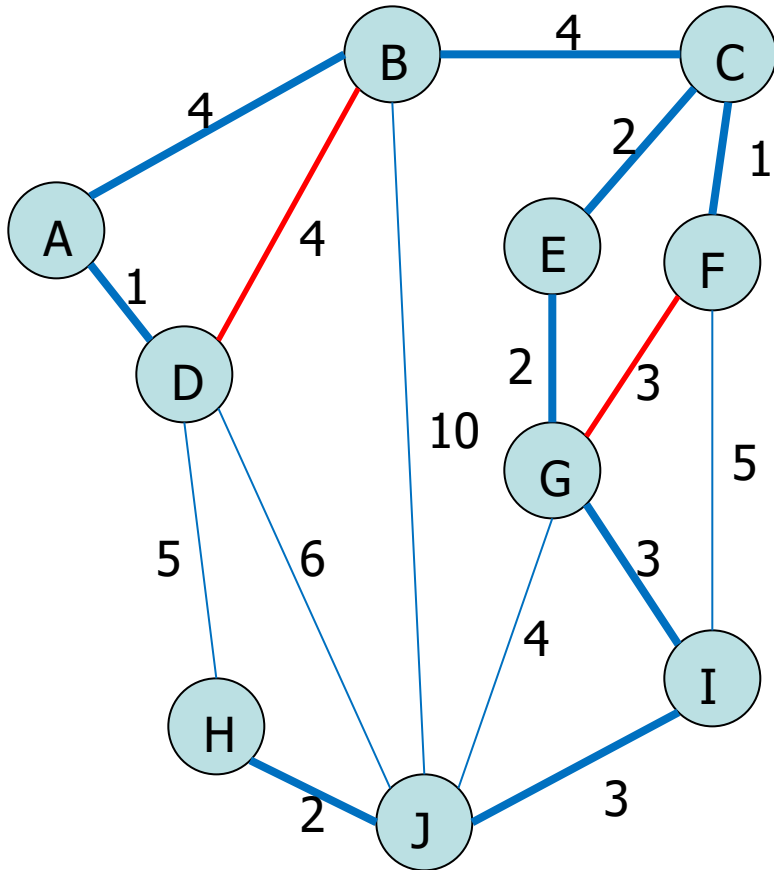


25-MST

30

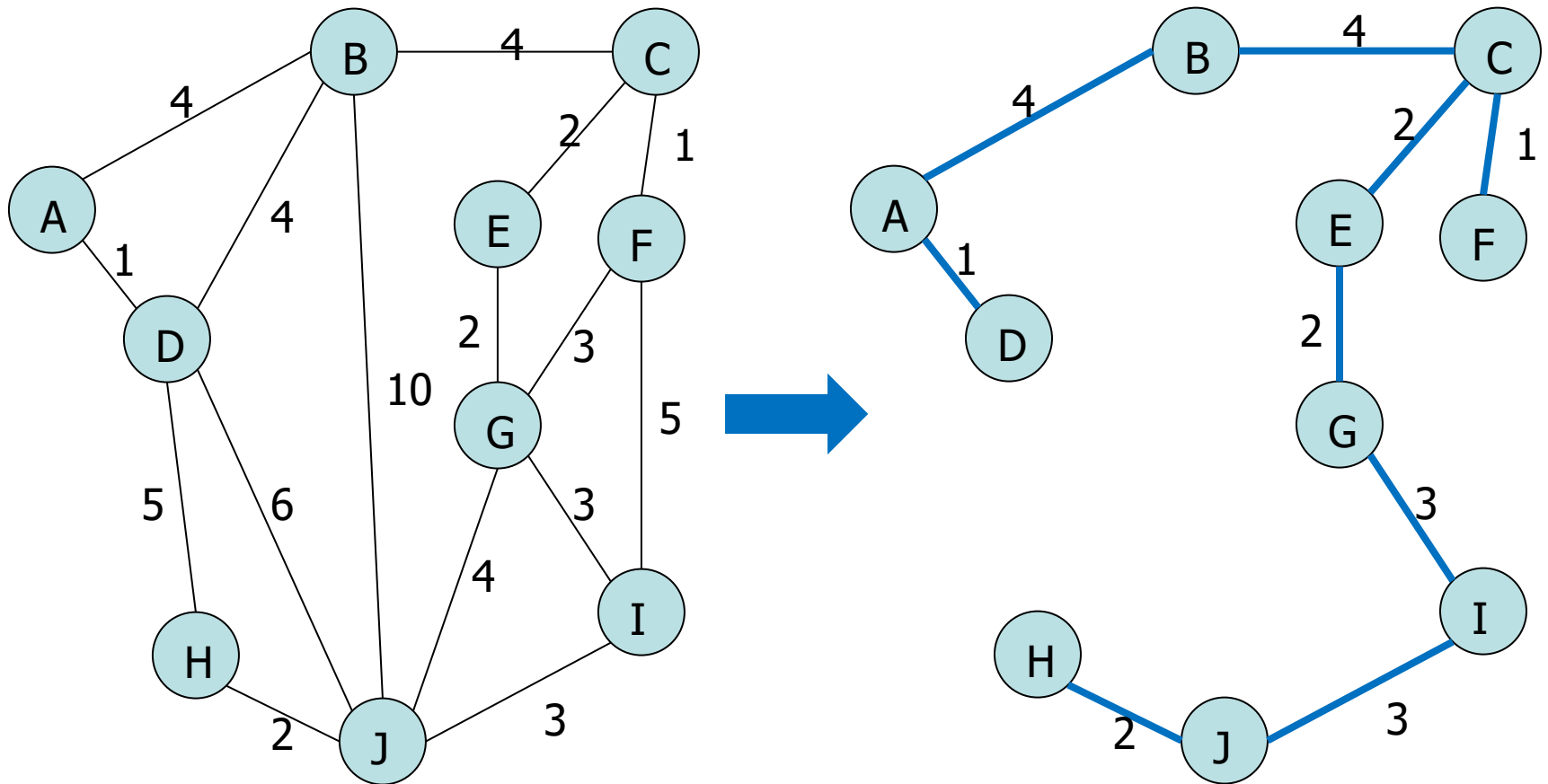
Kruskal's Algorithm – Example

- Add Edge



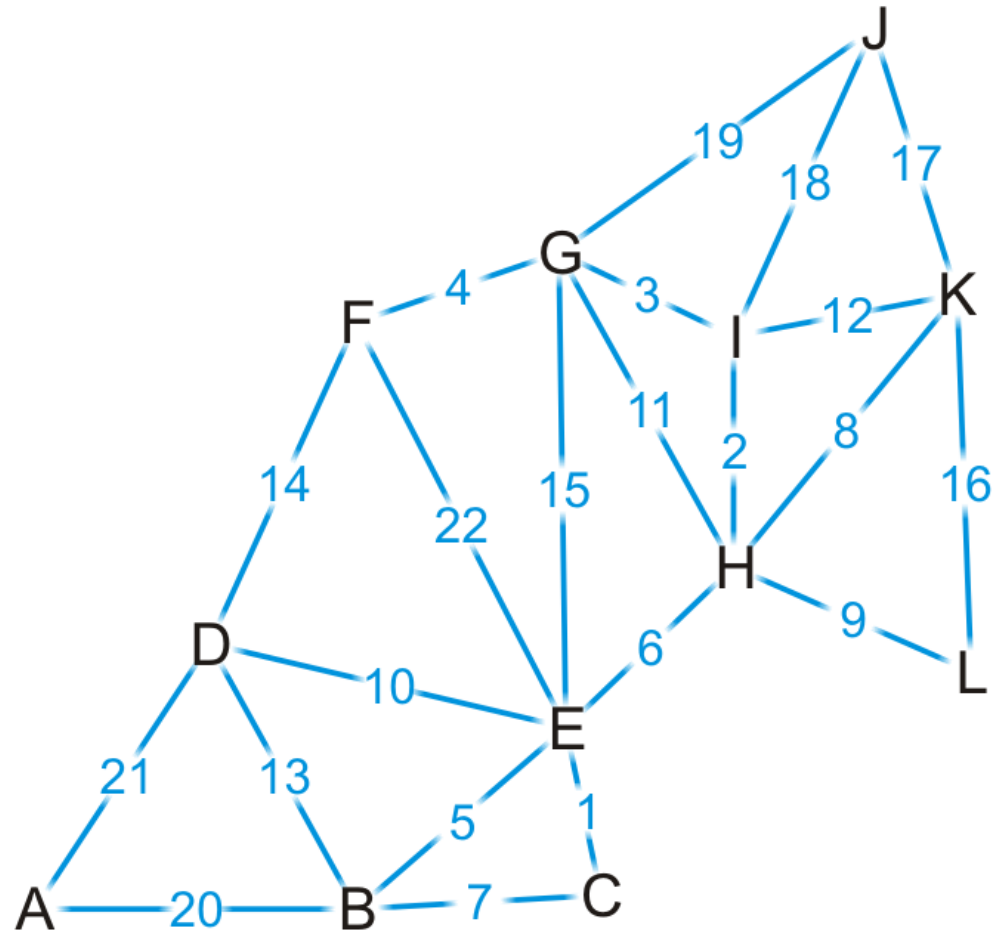
Kruskal's Algorithm – Example

- Minimum spanning tree



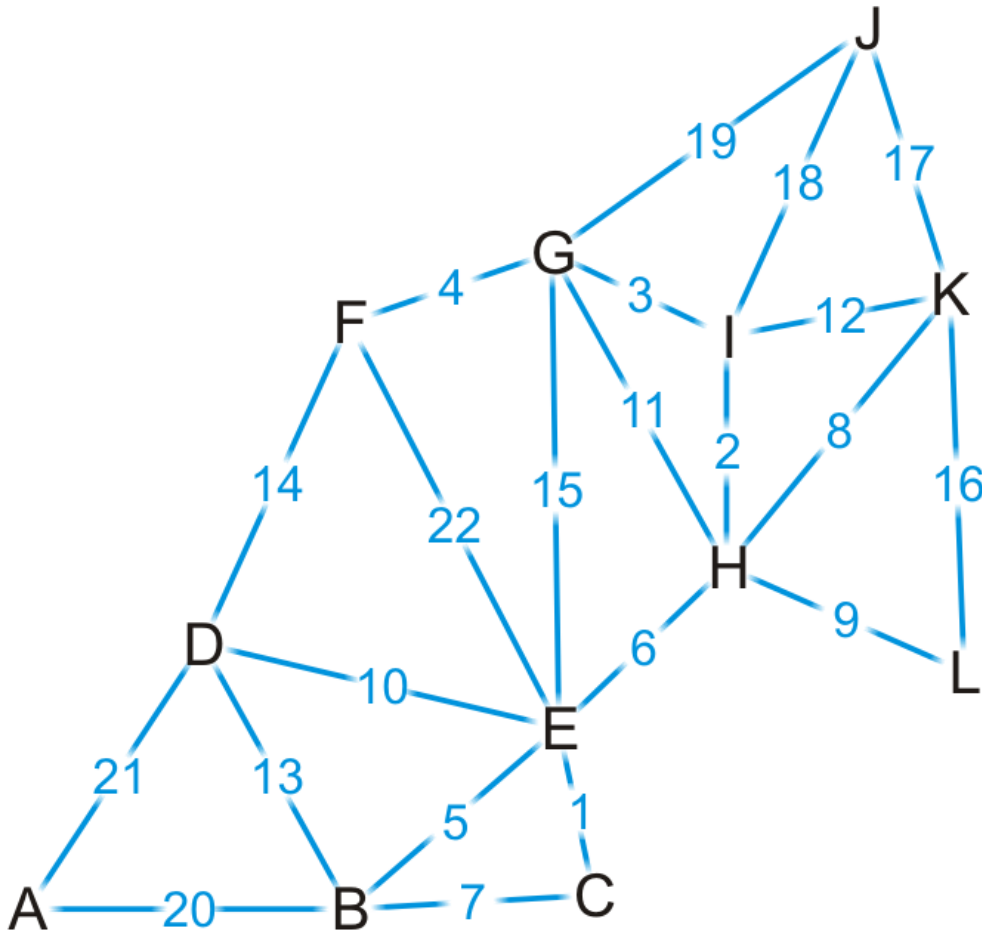
Kruskal's Algorithm – Example

- Complete graph



Kruskal's Algorithm – Example

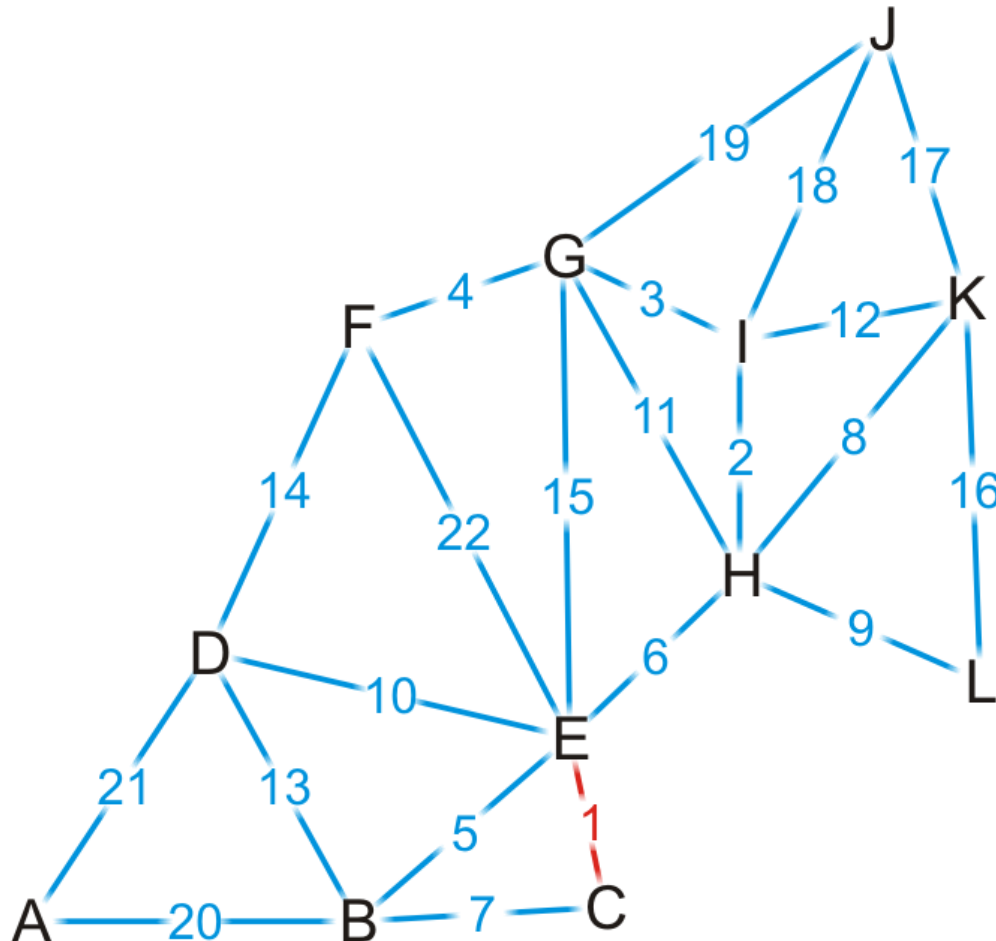
- Sort edges based on weight



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

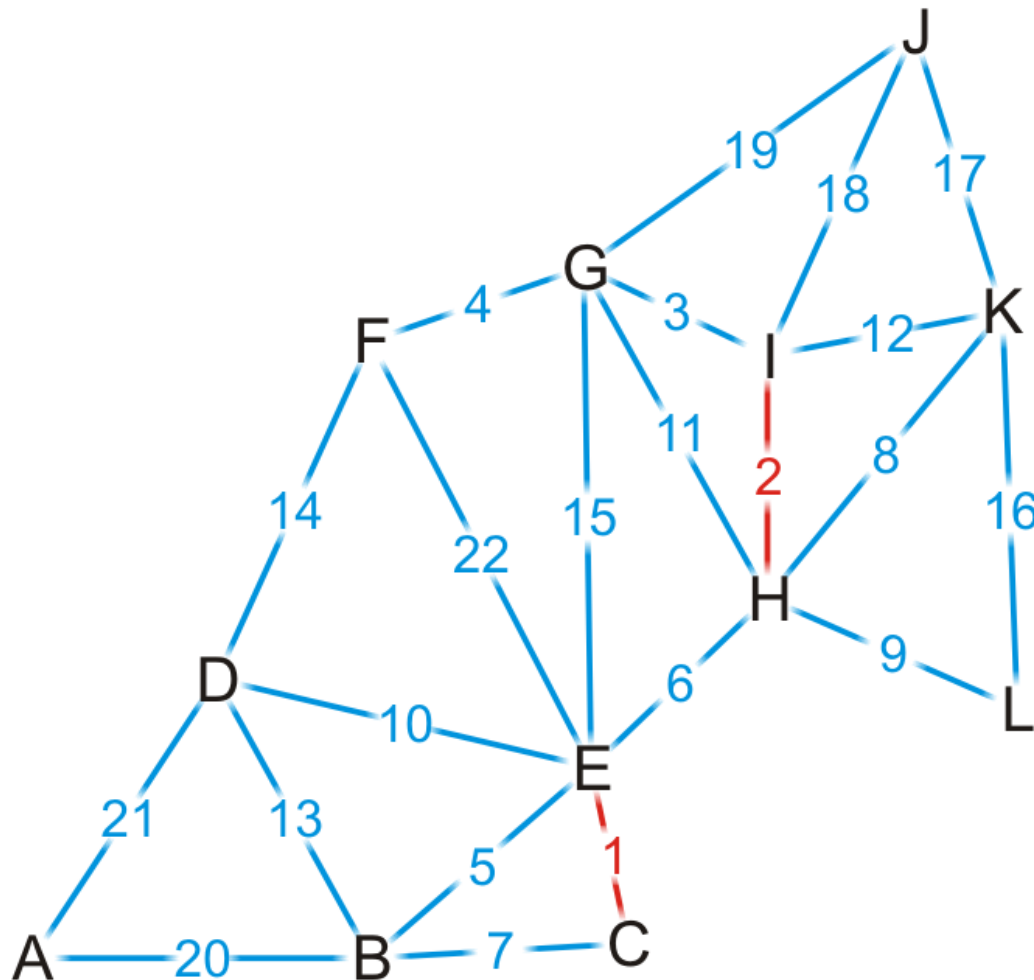
- We start by adding edge {C, E}



→ {C, E}
{H, I}
{G, I}
—
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

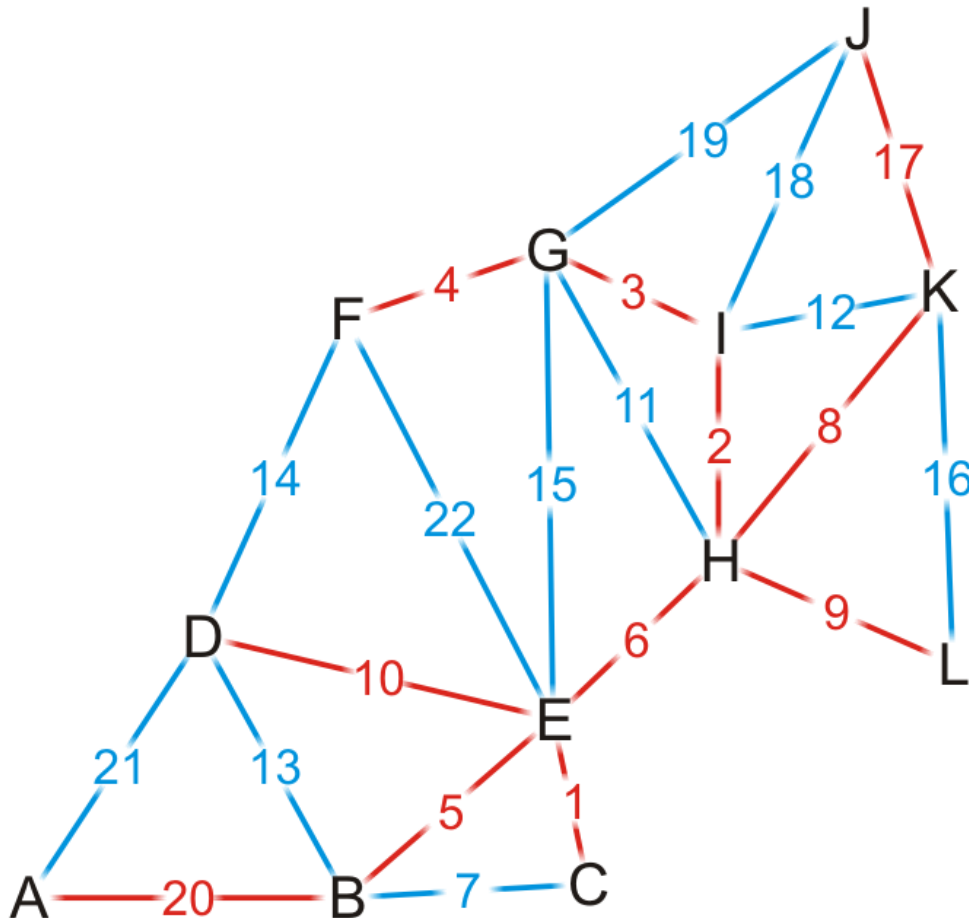
- We add edge {H, I}



→ {C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

Kruskal's Algorithm – Example

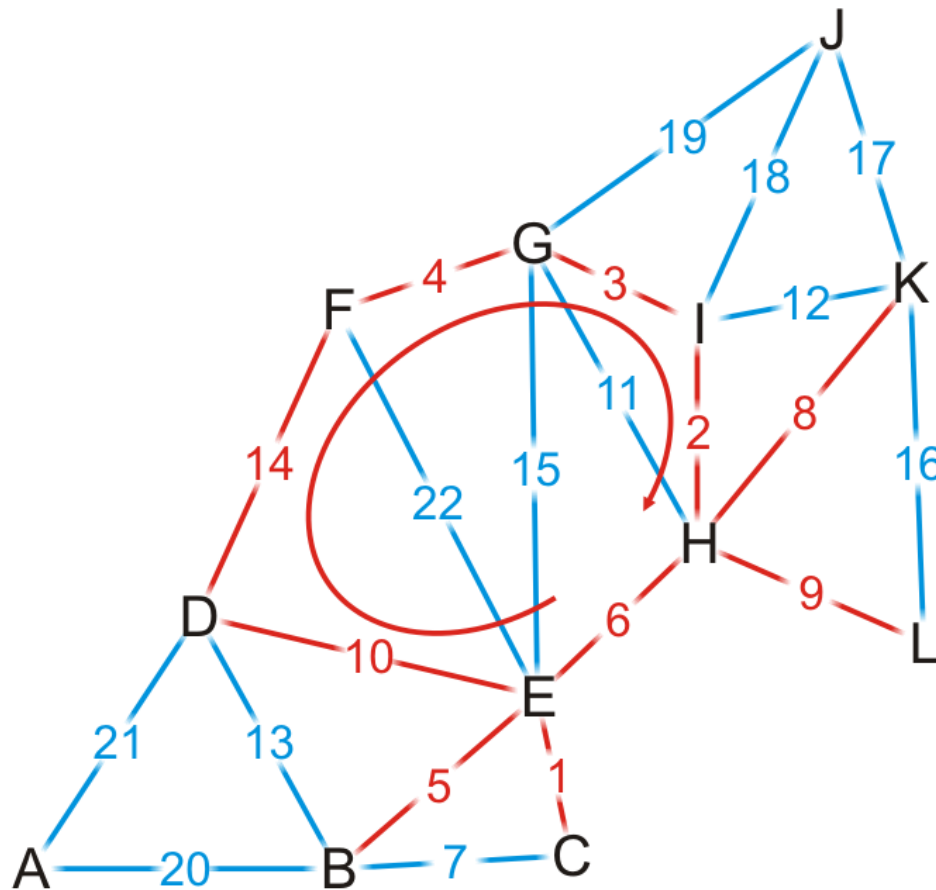
- Having added {A, B}, we now have 11 edges
 - We terminate the loop
 - We have our minimum spanning tree



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
→ {A, B}
{A, D}
{E, F}

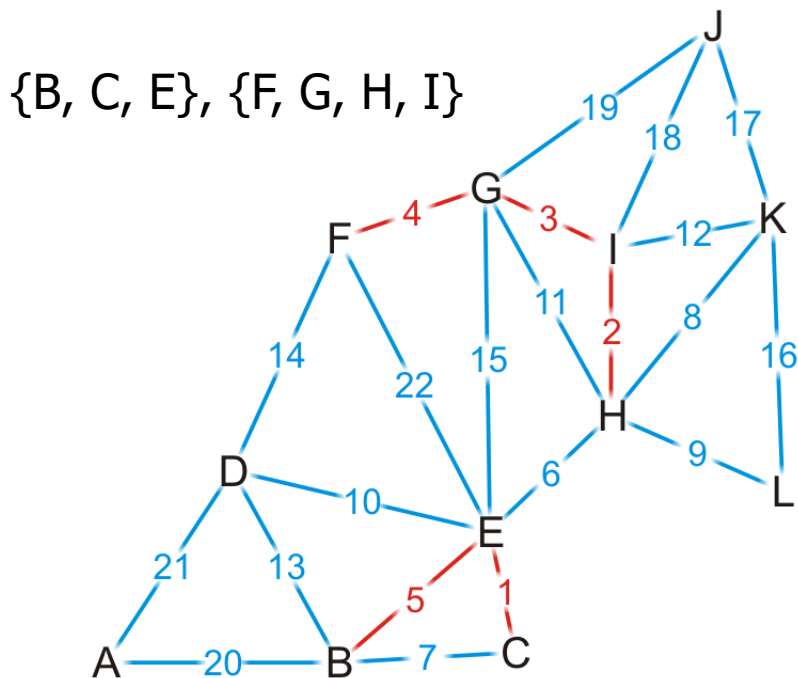
Detecting a Cycle

- To determine if a cycle is created, we could perform a traversal
 - A run-time of $O(|V|)$



Detecting a Cycle – Disjoint Sets

- Consider edges in the same connected sub-graph as forming a set

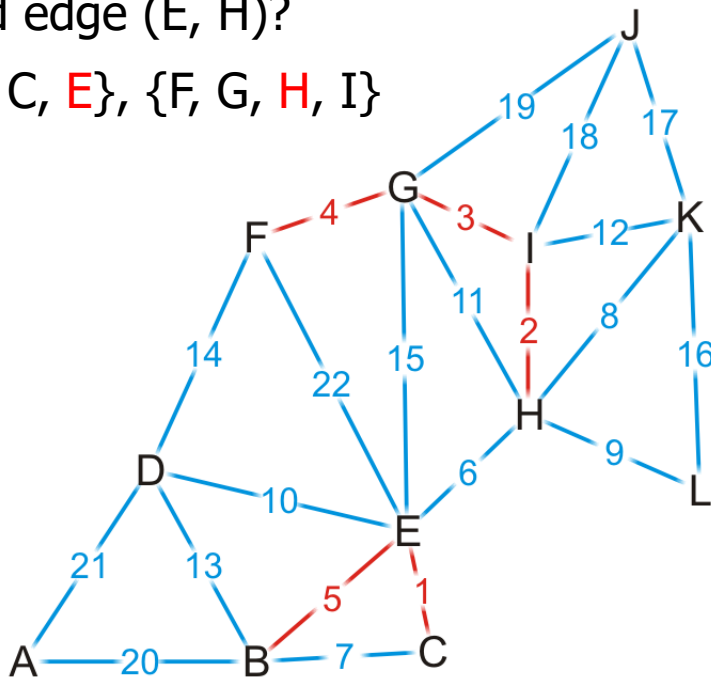


Detecting a Cycle – Disjoint Sets

- Consider edges in the same connected sub-graph as forming a set
- If the vertices of the next edge are in different sets
 - Take the union of the two sets

Add edge (E, H)?

{B, C, E}, {F, G, H, I}

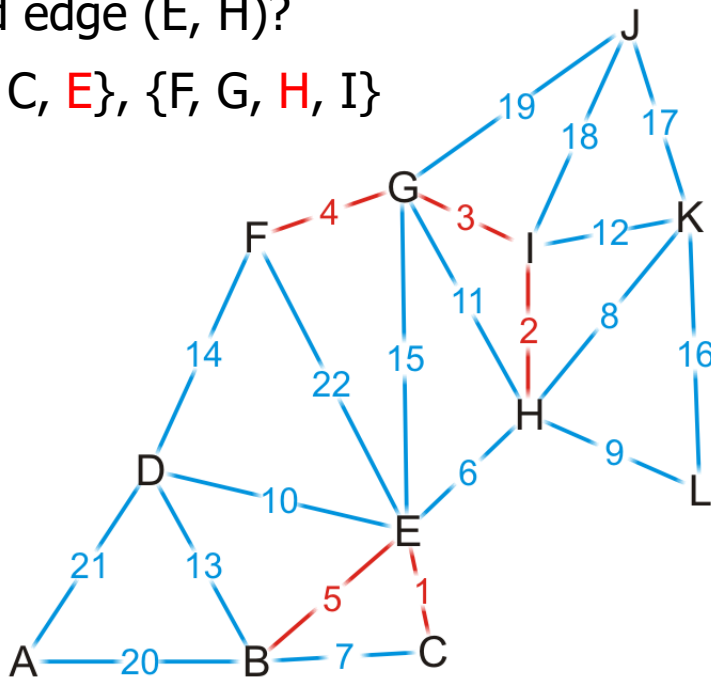


Detecting a Cycle – Disjoint Sets

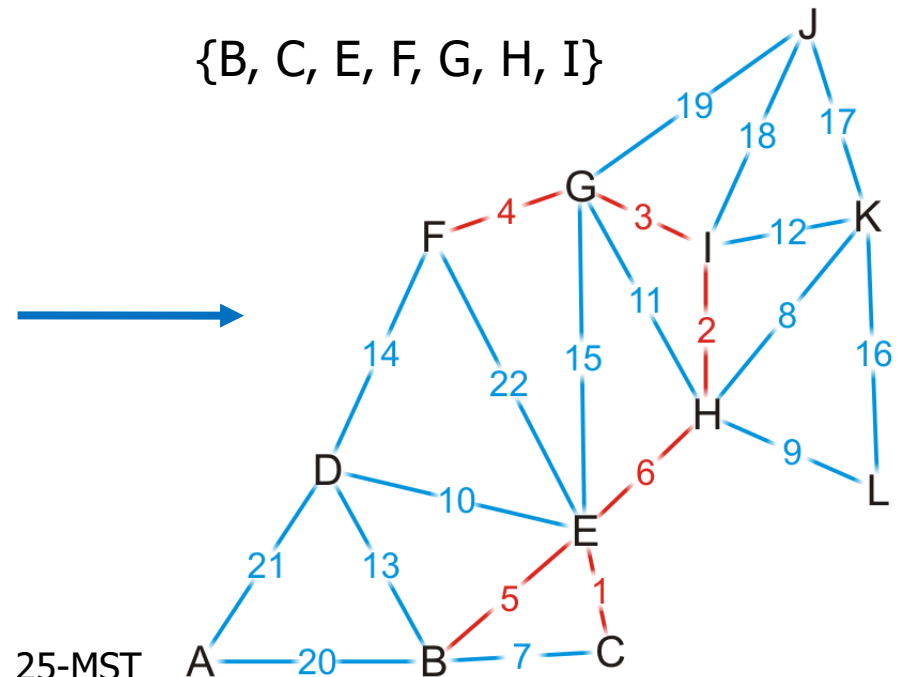
- Consider edges in the same connected sub-graph as forming a set
- If the vertices of the next edge are in different sets
 - Take the union of the two sets

Add edge (E, H)?

$\{B, C, \textcolor{red}{E}\}, \{F, G, \textcolor{red}{H}, I\}$



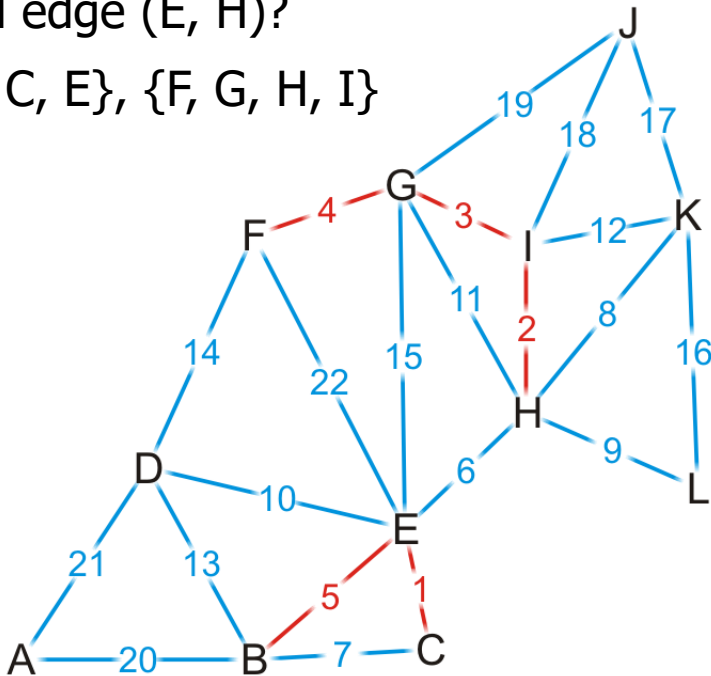
$\{B, C, E, F, G, H, I\}$



Detecting a Cycle – Disjoint Sets

- Consider edges in the same connected sub-graph as forming a set
- If the vertices of the next edge are in different sets
 - Take the union of the two sets
- Do not add an edge if both vertices are in the same set

Add edge (E, H)?

 $\{B, C, E\}, \{F, G, H, I\}$  $\{\mathbf{B}, \mathbf{C}, \mathbf{E}, \mathbf{F}, \mathbf{G}, \mathbf{H}, \mathbf{I}\}$ 